

UPPSALA
UNIVERSITET

IT mBM 25 009

Degree project 30 credits

September, 2025

Unsupervised Microfluidic Chip Anomaly Detection in Image Data

Leveraging Methods Based on Machine Learning Features: PatchCore, PaDim

Pengmei Hao



Master's Programme in Image Analysis and Machine Learning



UPPSALA
UNIVERSITET

Unsupervised Microfluidic Chip Anomaly Detection in Image Data

Pengmei Hao

Abstract

In industrial productions, anomaly detection has always been important to maintain quality consistency. Medical industrial production requires rigid production standards, as each item serves diagnostic purposes for patients. Anomalous products slipped into passed ones, can be detrimental.

Quality assurance which applies at the end of production is meant to detect and discard anomalous products. As production processes are not fully deterministic, production variants can be decided pass or fail under different circumstances. Subtle defects, can be overlooked, while new unprecedented anomalies remain undiscovered.

Based on these observations, we apply unsupervised machine learning techniques to combat this tricky and challenging issue. This work leverages the state of the art unsupervised machine learning methods including PaDim, PatchCore, and optimizes these methods under real-time industrial production settings. Anomaly scores predicted by both models are evaluated against the image labels, where PatchCore achieved 0.7691 in AUCROC and PaDim achieved 0.5734. Further, we developed a detailed rule sorting mechanism on PatchCore's heatmap predictions, and managed to sort out 21% chips(401 chips) from the full testing set of 1904 images. The sorting mechanism on this subset of chips achieved 88.78% accuracy(159 true positives, 197 true negatives, 42 false positives and 3 false negatives). Processing 21% of the manual examination workload in only 7 seconds per image, the proposed solution accurately detects subtle defects and has strong potential to boost efficiency.

Faculty of Science and Technology

Uppsala University, Place of publication Uppsala

Supervisor: Petter Ranefall and Gustav Jonsson-Glans Subject reader: Joakim Lindblad

Examiner: Nataša Sladoje

Acknowledgement To Nora, Jessica and Tsion, my team members for the preceding project, for the busy and wonderful time we worked together which paved the path for this project. Thanks to Petter and Gustav, my supervisors, for the good pacing and extra care of my project during summer. Sincere gratitude to the administration teams that supported me and made this happen.

Disclaimer Due to confidential reasons, a number of the figures illustrating the chips are masked off so that the readers are able to access without non-disclosure agreements.

Use of Generative AI Tools In this project, large language model(**`GPT-4.0`**) was employed as a supportive tool during both the implementation and documentation phases. It primarily assisted with generating code components, interpreting error messages, contributing to conceptual explorations, paraphrasing text passages, and creating LaTeX-formatted elements. Any AI-generated content was carefully examined and further reformatted by the author.

Contents

1 Introduction **1**

2 Purpose, Aims and Motivation **2**

2.1 Delimitations 2

3 Background **4**

3.1 Current project specific solution at the company 4

3.2 Unsupervised Machine Learning in 2D Images 5

3.2.1 Reconstruction Based 5

3.2.2 Embedded methods 5

3.2.3 Flow Based 6

3.2.4 Knowledge Distillation 6

3.2.5 Synthetic Anomaly 6

3.2.6 Hybrid Methods 6

3.2.7 Adaptations with Popular Models 6

4 Method **7**

4.1 Pre-processing (Point Cloud Alignment, Region of Interest Segmentation) 7

4.1.1 Point Cloud Alignment 7

4.1.2 Measurement of Chip Structures 10

4.1.3 Denoising and Pixel Error Handling 11

4.2 Unsupervised Machine Learning — Embedded Methods 14

4.2.1 Datasets and Model Configurations 16

4.3 Heatmap Normalization and Processing 18

5 Results and Discussion **23**

5.1 Training and Testing Efficiencies 24

5.2 Qualitative Comparisons Between PatchCore and PaDim 24

5.3 Anomaly Score Statistical Evaluations 26

5.4 Challenges in Differentiating Anomalies 28

5.4.1 Major Causes of False Alarms 28
5.4.2 Difficult Anomaly Border Cases 30
5.5 Rule Based Sorting — PatchCore 32
5.5.1 Rule to Sort Out Bad Chips 32
5.5.2 Rule to Sort Out Good Chips 36
5.5.3 Sorting Summary 41
5.5.4 Contradictions with Annotation 42
5.5.5 Newly Found Deformation Marked by the Model 43
5.6 Model Efficiency Experiments Under Reduced Dataset — PatchCore . . 44
5.7 Limitations of Model Learning 47

6 Conclusions **47**

7 Future Work **48**

A Appendix **55**
A.1 Double Hough Missing Chips 55
A.2 Row and Column Width Distribution 57
A.3 Exploration on Heatmap Visualizations 60

Acronyms

HIP Short for HIP BIT, is the plastic part that couples with the silicone part, forming the chip. 28, 32, 33, 35, 36, 47, 49, 50

RANSAC Random sample consensus is an iterative method to estimate parameters of a mathematical model from a set of observed data that contains outliers. 11, 12, 40

Shim The mould for producing the HIP BIT. 29, 36, 38

SOR Statistical Outlier Removal is a way to remove outlier points by their average distance to their neighbours. 11

1 Introduction

Microfluidic chips are widely used for biomedical and chemical applications, where fluids can be tested in a controlled and precise manner, as explained by Gharib et al. [1]. The microfluidic chips used in this work were created by imprinting micro channel structures in silicone material and then bonding these structures on top of a small flat surface, creating closed pathways where fluid can flow with high precision. In order to ensure correct testing of the fluids, it is essential that as many defective microfluidic chips as possible are detected and discarded before being used.

For precise representation of the microfluidic chips, 3D data was obtained using a line confocal sensor LMI FocalSpec [2] that scans an object's structures and creates high-resolution point cloud data. Each point cloud in our dataset contains four chips positioned in a row next to each other as can be seen in Figure 1. The point cloud data format enables the collection of complex information such as geometric relationships, which could be greatly beneficial for the detection of defects. The collected point clouds were large (one file, showing 4 chips: 0.5GB, with each chip around 8,000,000 points), making the total dataset 300GB with over 2,000 chip images. Along with each image a binary label is provided, indicating if the chip is a pass or fail.



Figure 1: Example of the original point clouds with four microfluidic chips in a row.

Known defects include scratches in different parts of the chips, holes, pipe breaks, chips with white stickers on top, channel blockage, channel connections, chips with mostly white points and debris. Examples of the known defects can be seen in Figure 2. An example of a few defects in one microfluidic chip can be seen in Figure 3.



Figure 2: Example of defects: scratch, pipe breakage, debris, channel connection and a chip with a white sticker on top.



Figure 3: Example of one chip with a collection of defects, red boxes are manually added for displaying defects.

2 Purpose, Aims and Motivation

The aim of this work was to explore methods for detecting deviations and defects in microfluidic chips which are represented as binary labeled point cloud data. This work focuses on the development of techniques for automatic detection of known defects that could potentially hamper the testing of the fluids, as well as exploratory analysis to detect features deviating from the majority. Our target is to verify the state-of-the-art machine learning anomaly detection methods and hopefully integrate effective ones into the production process.

Research questions:

- What are the state-of-the-art methods on anomaly detection with unsupervised machine learning?
- Are there any overlooked anomalies in the dataset? Are there any undiscovered anomalies?
- Does the methods work well, and how can we incorporate them into productions?

2.1 Delimitations

The FocalSpec imaging has brought a large number of floating pixels below and above the chips as shown below in Figure 4. Which are results of reflection or optical interference.

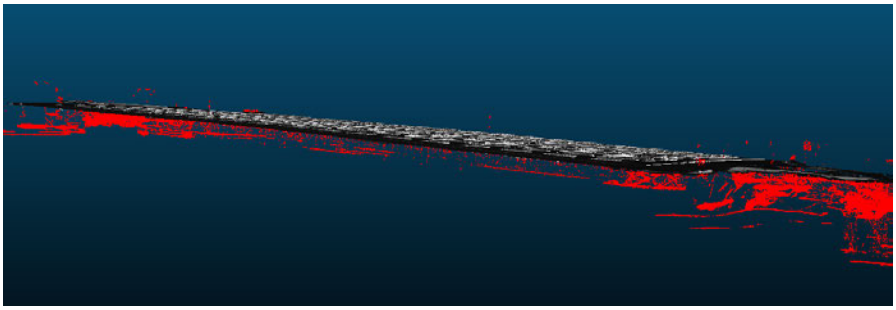


Figure 4: Disturbance in point clouds marked as red.

These disturbances causes great distraction for the anomaly detection models as the projected 2D images also include these pixels when seen from above, as shown in Figure 5.

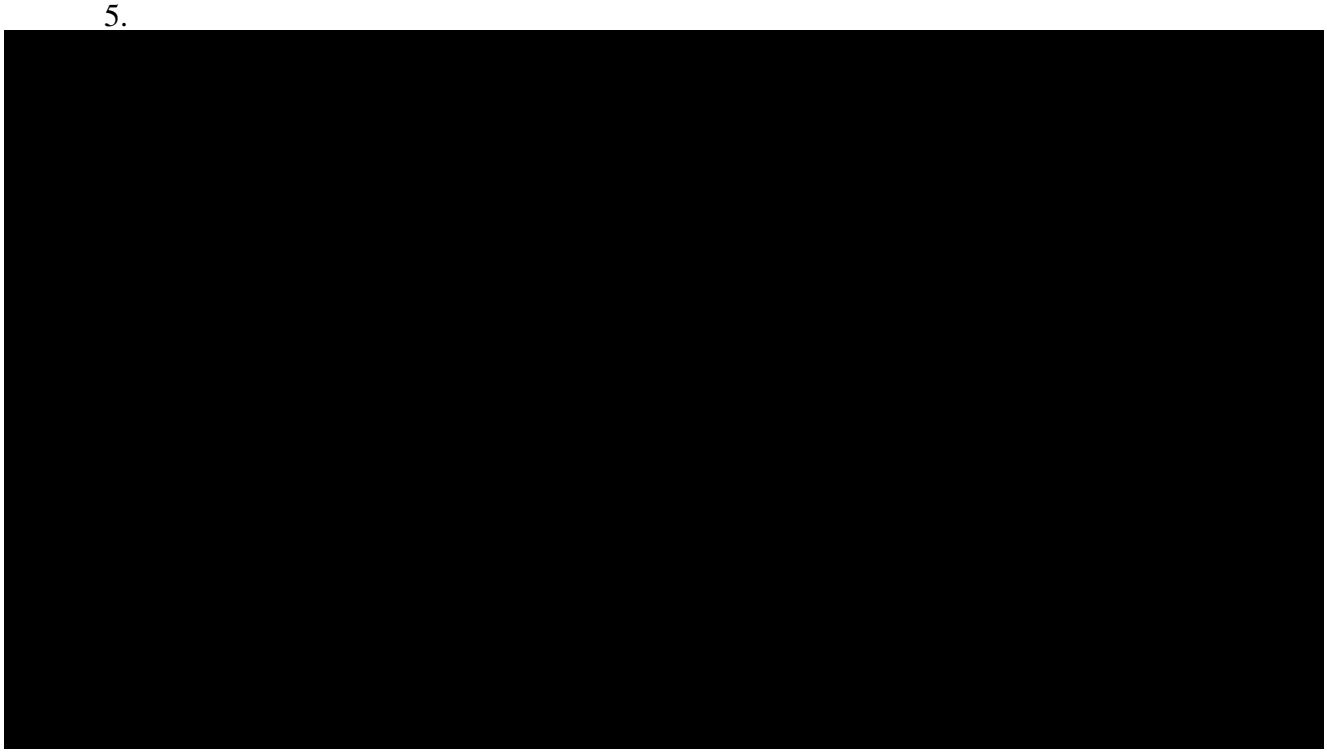


Figure 5: Wide spread of individual pixels seen from above.

The models, in general, are vulnerable to abundant disturbances like these. Both feature based models like PatchCore and PaDim give out a number of false alarms on these pixels. As a result, we have taken measures to filter out the disturbance pixels in the 3D point clouds, as well as using mean filters to fill up the disturbance positions in 2D. Section 4.1.3 explains in detail how the denoising measures were applied.

Operators in production typically disregard those unknown pixels as they are comparatively small and only block a small fraction of the structures. On the other hand, the company is also actively searching for solutions to improve imaging quality. Therefore we decide to use the filtered images for machine learning methods.

3 Background

The company manufactures microfluidic chips, which are used in healthcare, patient diagnostics, and medical research. It also produces related apparatus designed to work with the chips, together forming a complete diagnostic system.

In productions, the chips are made from [redacted] which is then captured by an instrument called FocalSpec to generate 3D point clouds. Currently the company's method relies heavily on manual inspection of the projected 2D images.

3.1 Current project specific solution at the company

The company has a tailored solution for anomaly detection in microfluidic chips. It integrates advanced imaging, classical image analysis, and manual inspection to identify defects, balancing automation with operator review to overcome the limitations of fully manual or automated systems.

High-Resolution Image Capture begins with capturing detailed bitmap images of microfluidic chips using the FocalSpec imaging system, which offers precise visualization of essential features such as the channels. 3D point clouds are then converted into 2D images, aligned with template matching for further processing.

Anomaly Classification Utilizing classical computer vision methods, each detected issue is classified into specific categories (e.g., missing wells, channel breaks) to simplify further analysis.

Bounding Box Annotation The identified defects are marked on the chip's image, creating an annotated error map. This visual representation in Figure 6 and Figure 7 helps operators quickly pinpoint areas that require further attention or investigation.

Operators review the annotated images helps operators to make the final decision on whether a chip passes or fails. To ensure thorough inspection, the system is designed to flag potential issues even if some are false positives, alarming all potential anomalies.



Figure 6: Template matching result, four potential defects were detected: three bounding boxes indicate error 4 (in rows [19, 20, 23]), and one bounding box indicates error 3 (in row[20]) - 3301-20-F-1

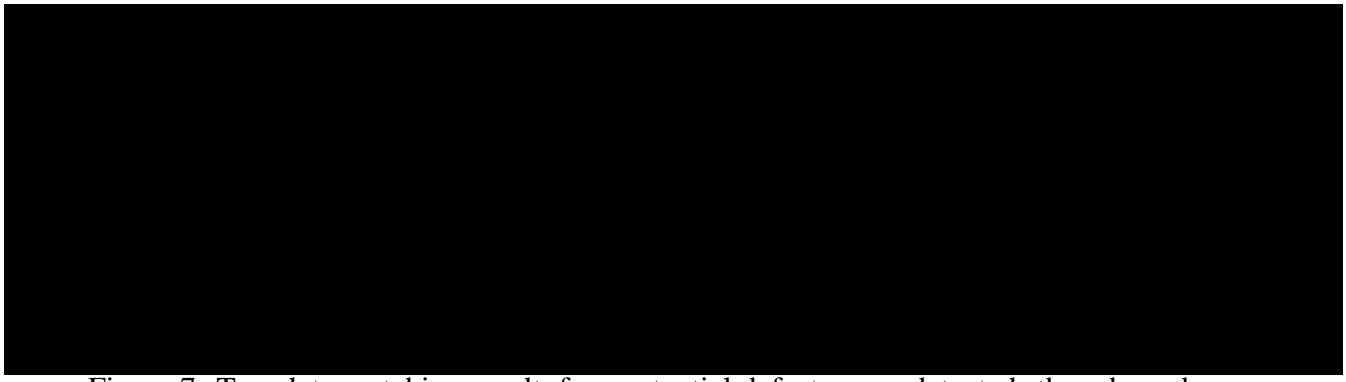


Figure 7: Template matching result, four potential defects were detected: three bounding boxes indicate error 3 (in rows [5, 10, 21]), and one bounding box indicates error 4 (in row[5]) - 3299-12-C-3

3.2 Unsupervised Machine Learning in 2D Images

Currently there are several different unsupervised machine learning ideas on anomaly detection, the ones that we touched upon and researched are:

3.2.1 Reconstruction Based

Early development of anomaly detection uses autoencoders[3]. By training the encoder on normal images, the difficulties in reconstructing bad ones are quantified as losses to indicate anomalies. This intuitive idea was soon further transformed into more complicated reconstruction processes like *student-teacher*[4] that uses 2 encoders learning from one another, *reverse distillations*[5] that trains a decoder to reverse the pretrained encoder or adversarial network in the form of GANomaly[6].

3.2.2 Embedded methods

Methods that exploit the features extracted by a pretrained backbone. No training phase involved as backbones are pretrained on ImageNet, such as: ResNet-18, WideResNet-50-2 and etc.

An early predecessor in this direction is the **SPADE**[7] method from 2020, which does a kNN search in the feature pyramid to differentiate anomalies.

PaDim(Patch Distribution Modeling)[8] method from 2021, unfold lower feature layers and concatenate them with higher layers, forming a feature vector for each patch. Then models vectors in corresponding locations by multi-variate Gaussians.

PatchCore[9] method from 2022, concatenates the feature layers, subsamples the patch vectors and saves them in a memory bank. Then does nearest neighbor search in testing. Publications built upon the memory bank idea is: CFA[10] which requires training.

HETMM[11] from 2024, which goes deeper into pretrained features, and does template matching on feature levels.

3.2.3 Flow Based

Methods that involve normalizing flow in reconstruction, like CFLOW-AD[12] and FastFlow[13] which utilizes the pretrained model and normalizing flow to differentiate anomalies.

3.2.4 Knowledge Distillation

Initial development is the essembled ”Uniform Students”[14], then STFPM[4] applied student teacher structure across different feature levels.

3.2.5 Synthetic Anomaly

Similar to reconstruction based but utilizing added synthetic anomalies on the normal images, taking advantage of ”known” ground truth as they are artificially added, and train the model to do segmentations as per the added anomaly. Prototype method is the DRAEM[15] and CutPaste[16] published in 2021. Methods that follow and develop are DRAEM+SSPCAB[17] and DRAEM+SSMCTB[18] which are inspired by attention mechanisms. Related methods are SimpleNet[19] which synthetic feature leveled anomalies and PBAS[20] that delves deeper into feature level anomalies. GLASS[21] is a new anomaly synthetic strategy with gradient ascend in feature space.

3.2.6 Hybrid Methods

Methods develop after 2023 are making innovations by combining preceding methods. EfficientAD[22] uses a student-teacher structure and an auto encoder, with a specific pretrained model.

DeSTSeg[23] utilizes synthetic anomalies in the student teacher model.

3.2.7 Adaptations with Popular Models

These methods make direct use of newly developed models to do anomaly detection, like Mamba[24], Diffusion Model[25], and Transformer[26].

These new adaptations are insightful but not as efficient as previous CNN based methods[24]. New models are creatively applied regardless of high computational demands. Recently claimed benefits of these models include multi-class anomaly detection[27](one model to detect all object categories) and few-shot anomaly detection[26](using only a few images for training).

4 Method

4.1 Pre-processing (Point Cloud Alignment, Region of Interest Segmentation)

Chips are cut from a row of four, after applying denoising and preliminary centering. We can see that the generated point clouds are not very well aligned as shown in Figure 8. Their centers are not aligned, and there are slight rotations of a couple of degrees which varies chip by chip, making it difficult to select a consistent region of interest for machine learning models. The projections in Figure 8 have been trimmed by selecting a range of x and y, but more detailed segmentation is not possible as further segmentation would possibly cut into the channels. The image corners, as well as the varying thin white peripheral lines, very often attract the attention of the models. Causing frequent false alarms.



Figure 8: A list of four chips without alignment.

4.1.1 Point Cloud Alignment

We solve the alignment problem starting by segmentation of the 3D point clouds. Point clouds of the channels are projected to 2D, then using Hough segmentation to acquire the circle centers.

First the 3D point clouds were flattened into 2D where only x and y coordinates were saved. The x and y coordinates from the 3D space were mapped into its corresponding position in the 2D pixel space. The 2D images were binarised and edges were detected using Canny edge detection [28]. An example of the extracted edges can be seen in Figure 9a.

Hough transform was used with the radius set to 27 pixels, found to be approximately the size of the circles. After filtering out circles close to each other and checking if they are within bounds, 128 circles per chip could be found, see Figure 9b.



(a) Canny edges. (b) Detected circles. (c) Circles in 3D space.

Figure 9: Canny edges and detected circles in 2D and 3D.

The segmentation method for the ”circles” in 3D is further optimized as a ”double-Hough”, to perform Hough transform again on the what is left after the ”first-Hough”, further collecting the occasional missing circles. As shown in Figure 10. Finally the detected 2D circles were used as masks for the 3D point clouds, where all points in the z direction were kept in Figure 9c. This optimized ”Double Hough” method is quite robust. Among 2,144 images, only 2 chips still have missing circles after ”double-Hough”. The details of those 2 chips can be found in Appendix A.1.



(a) Result of first Hough (b) Result of second Hough (c) Combined Hough segmentation

Figure 10: Double hough process.

The ”Double Hough” produces full 128 circles segmentation in the 3D point cloud. To enable sorting of the circles before saving them, a grid of the combined circle centers are grouped using k-means algorithm to form a ”matrix” of 16×8 circle centers. The 8 columns were found by applying k means with k set to 8 on the x values in the 2D space.

Leveraging the circle centers in 3D positions, we are able to acquire geometric features such as the slope as well as the center of the chip. By taking mean of the 4 circle centers symmetric to the centers, such as 8th row 4th column, 8th row 5th column, 9th row 4th column and 9th row 5th column, the center of the channels can be acquired. Then, by picking the most distant 2 circle centers and fit to a straight line, the chips rotation angle is exactly the line's slope. The 3D point cloud is precisely aligned by utilizing these 2 features. Chip-wise aligned process is illustrated in Figure 11.



Figure 11: Chip alignment by geometric features.

After alignment, the uninteresting regions beyond the channels are removed, by extending from mean circle centers of the outer circles. Horizontally by 1.1 Hough radius, vertically 1.3 or 1.7 Hough radius depending on top or bottom. This way only points within the edged circles' rims will be kept, as illustrated in Figure 12.



Figure 12: Cutting off uninteresting regions by extending from circle centers.

Finally, the chips are aligned very consistently, see Figure 13. The homogeneity of chip alignment is ideal for machine learning.



Figure 13: A list of four aligned segmented chips.

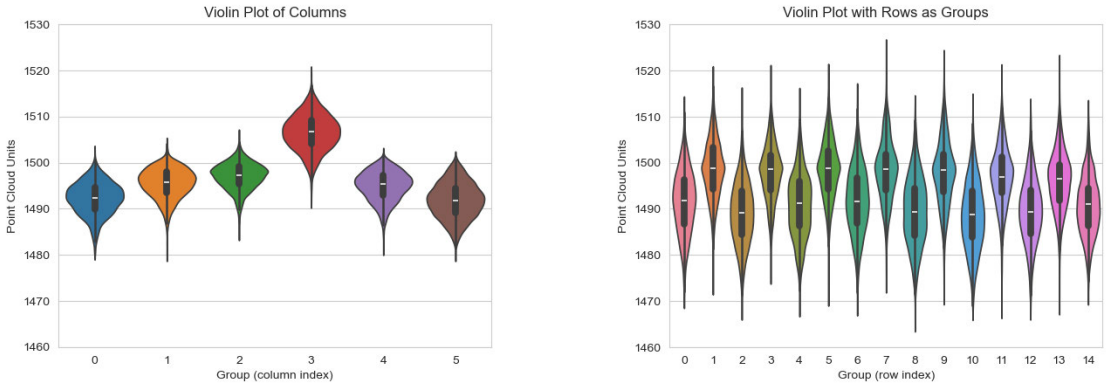
4.1.2 Measurement of Chip Structures

With the point clouds well aligned, the row widths and column widths are easily acquired by subtracting the circles' center's x or y values across the rows or the columns. Rows and columns are numbered as per Figure 14.



Figure 14: Illustration of rows and columns.

As shown in Figure 15, we can see that the column width is less homogeneous than the row width. Across the columns, the distance between the circle centers varies, but column1 and column4, column0 and column5 are approximately symmetric, echos with the overall vertical symmetrical structures. Column3 and column2 not only have different distributions, but also have a difference in the mean, with column3 thicker by 10 point cloud unit distances. Which means column3 is 0.6% thicker than column2. While other symmetrical columns only have a difference of 0.04% and 0.03%.



(a) Violin plot of width of columns.

(b) Violin plot of width of rows.

Figure 15: Distribution visualization of row and column width of the full testing set (1904 chips).

The column and row lengths are also plotted along the time axis, to see how much the distance between the rows and the columns vary along production. However, factors such as the work distance between the lens and the chip may cause measurement issues. The measurement plots can be found in the Appendix A.2 lest the reader to be interested.

4.1.3 Denoising and Pixel Error Handling

After point cloud alignment, we first denoise in 3D by RANSAC where RANSAC is used to recursively fit the points into the chip plane and points outside a distance threshold is considered noise. Then SOR filter is used to further remove floating points. The removed pixel errors are displayed in Figure 16, with most of which below the chip plane. The red points(around 300,000) are those detected and removed by RANSAC and the yellow points(around 20,000) are those removed by SOR filters.



Figure 16: 3D noise removal visualizations, chip is displayed against a blue-black transition background. Red points are disturbances removed by RANSAC, yellow points are disturbances removed by SOR filters.

Occasionally a floating debris would also be removed by the same process. An example of debris removal is shown in Figure 17.



3D close view before denoising. On the right shows the debris being selected as outlier by RANSAC. A hole of the background color can be seen from the side views due to the debris blocking the camera.

The debris blocks the top view partially, and when removed leaves nothing but a hole in its blocking position, as shown in Figure 18. The original information cannot be recovered as they were never recorded.



background color.



Figure 19: Hole seen from above due to removal of pixel error.

Same thing can be said for pixel errors. For the abundant floating pixels,the same process takes place but with a much smaller scale. Smaller clusters of points above and below the chip will be removed leaving numerous holes, which can be concerning when they are blocking the channels like in Figure 19.

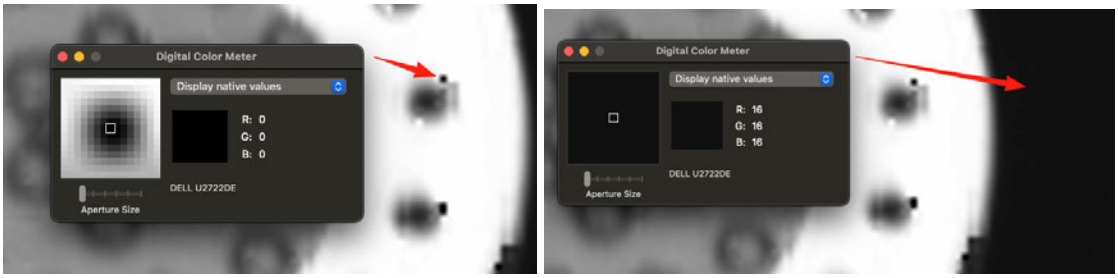


Figure 20: Color value of the holes and the background(in 2D).

Holes big or small are voids in the 3D planes. In projection to 2D the image is filled using RGB values from the 3D point clouds. When there is no point at the positions, it would remain unfilled with 0 intensity. This helps to differentiate those points from the background because the background value is usually around 16-18 like on the right of Figure 20. Then we applied median filters of [5,5] to 0 value pixels for three iterations, this way only the small holes will be filled and the large holes will shrink a bit. The idea behind this is to preserve the ”unknown” but in the same time filling up tiny holes

that are impossible to be anomalies. An example of which is shown in Figure 21. On average 90% of the 0 valued pixels are filled by this process.

Figure 21: Median filter filling on the projected 2D image(before and after).

4.2 Unsupervised Machine Learning — Embedded Methods

Our research is focused on testing and optimizing 2 major embedded methods, PatchCore and PaDim on the processed 2D images projected from the 3D point cloud.

During training, both models do feature extraction using pretrained backbones(ResNet, EfficientNet, WideResNet and so on), and build a feature ”block” structure inspired by the SPADE[7] method. Backbone model's intermediate layers are connected by upsampling and concatenation, as shown in Figure 22. The connected patch features block is further exploited by PaDim and PatchCore methods.

Unlike classical machine learning methods which has a standard training phase, both methods work on the extracted intermediate features for anomaly predictions. The backbone models' final output is disregarded.

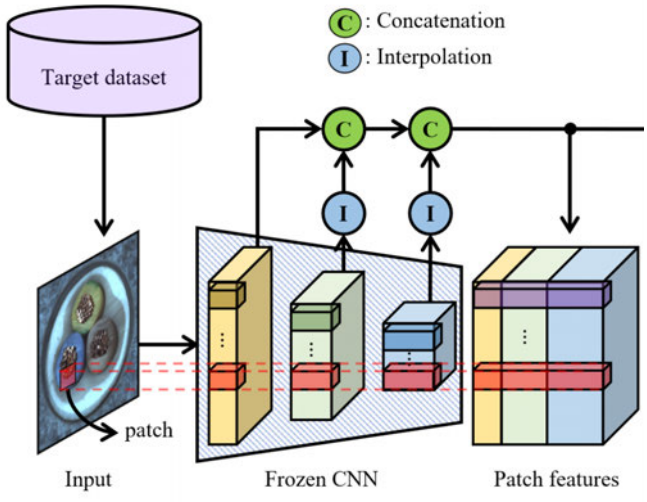


Figure 22: Feature extraction and ”block” building illustrated in the CFA[10] publication.

For PatchCore, feature extraction produces for each image a feature block. After 2 rounds of adaptive average pooling, $H \times W$(the patch grid dimension, $H \times W =$

101, 235) numbered feature vectors of target embed dimension were coreset subsampled by $p\%$. The process starts by randomly select 10 feature vectors, say $c_1, c_2, c_3, ..., c_{10}$, then for each selected vector c_j calculate its distance to other feature vectors x_i extracted from the backbone:

$$d_j = \min_{i=1,...,101235} \|x_i - c_j\|_2 \tag{1}$$

Select the new c_j into the original set of 10 as the x from $\arg\max_j d_j$, iterate until the number of vectors in set reach 506(p=*0.5%* of 101,235). Store these vectors into the memory bank, and repeat this max-min process with next training image, expanding from 506 vectors to 1012 vectors and so on till $N \times 506$ vectors. In our case $N = 50$ as we are having 50 images.

Then, during inference the testing images are compared with the feature vectors in the memory bank. Finding features-wise the nearest neighbor in the memory bank of $N \times 506$ vectors and mark the patch grid by ℓ_2 distance for each pixel(patch). Illustration of this process is shown in Figure 23.

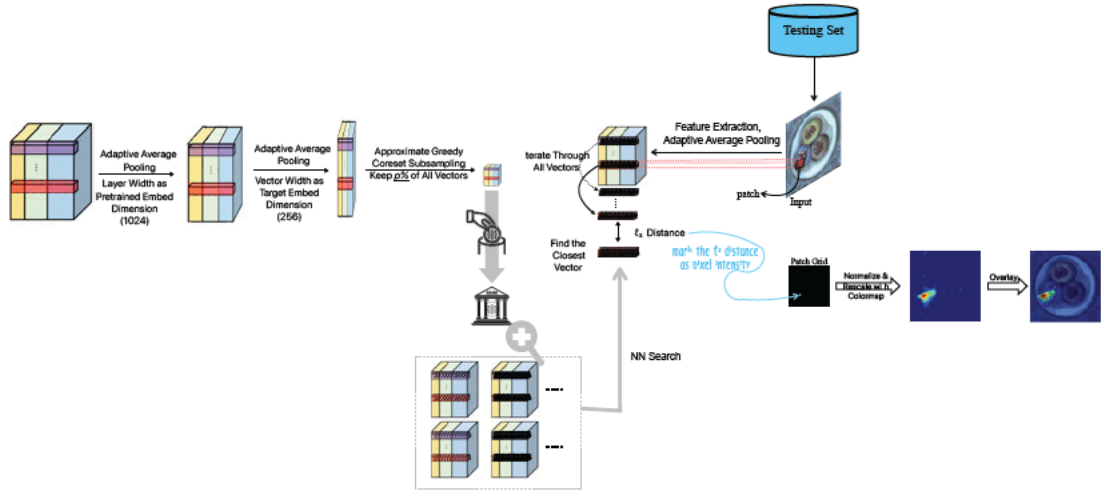


Figure 23: PatchCore testing process.

On the other hand, PaDim unfolds and concatenates channels from the layers. The distribution of the feature vectors is then modeled by a multivariate Gaussian. During inference, features are extracted from each input image as in training, and the patch-wise Mahalanobis distance to the modeled Gaussian distribution is evaluated. Finally, the distance value is registered in the patch grid. Process of which is shown in Figure 24.

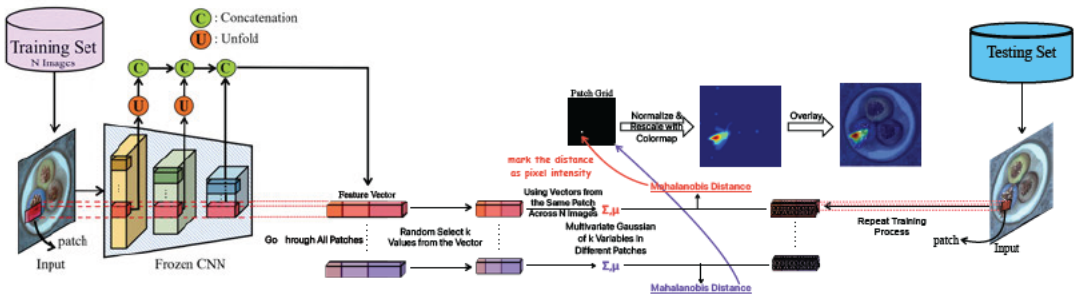


Figure 24: PaDim testing process.

The patch-wise multivariate Gaussian is calculated via the formula:

$$\mu = \frac{1}{N}\sum_{n=1}^{N} x_n, \qquad \Sigma = \frac{1}{N-1}\sum_{n=1}^{N}(x_n - \mu)(x_n - \mu)^T + \epsilon I$$

where N is the number of training images, and x is the feature vector. The Mahalanobis distance between the testing patch and the modeled Gaussian is:

$$M(X) = \sqrt{(x-\mu)^T \Sigma^{-1}(x-\mu)}$$

4.2.1 Datasets and Model Configurations

In academia, anomaly detection methods are often evaluated on the MVTec datasets, including 15 item categories and over 5,000 images in total. An example testing image of the "bottle" class is shown in Figure 25.



Figure 25: An anomalous example with annotated ground truth.

The MVTec dataset has also brought in some limitations. The resized input(256*256) and pixel-wise AUCROC evaluation has become a prevalent standard for model evaluation. In real-world production, well-annotated anomalous images are seldom accessible. As the possible type of anomalies is not fully known, it is not practical to assess anomaly segmentation on pixel level. Image resizing, while minimize computational load, is only practical when the anomaly is relatively large and visible. In our project there is an abundance of anomalies only 20*20 in a full image of 2048*3178, resizing would end up filtering out these small defects, and is therefore not applicable.

In conclusion, we have trained and tested our models on image level with no resizing. As a result, different model's strength and limitations widely differ from what is claimed in publications. Our project also benefit largely by tuning and adjusting the solution as per our dataset. Details of the datasets can be found in Table 1.

Table 1: Dataset details.

Datasets	Number of Good Images	Number of Bad Images	Image Size
Complete Dataset	1738	406	2048*3178
Training Set	50 or 100	0	2048*3178
Testing Set	1498	406	2048*3178

As you can see, the training set is fairly small compared with the full dataset of 2142 images. Feature extraction in both methods are very RAM demanding, and our large image input has made it even worse: each image is over 7MB. In the end we settled with using 100 images for PaDim and 50 for PatchCore. In both methods an attempt to double the training set has ended with memory overflow.

The two methods share a number of similarities, such as backbones, concatenated layers, vector length which is actually the channel length of the layers and etc. The details can be found in Table 2, where all adjustable hyperparameters are bold fonted. Hyperparameter selection were meant to prioritize performance(anomaly localization), so the selected configurations were compromised gradually from the default settings. The configurations in Table 2 are presumably the best, any increase in the hyperparameters will cause memory error or delay testing speed greatly. For PaDim, a small subsection (50) of the feature vector is selected randomly, corresponding to the "k" in Figure 24.

Table 2: Comparison of PatchCore and PaDim configurations. Bold entries indicate adjustable parameters.

	PaDim	PatchCore
Backbone	ResNet-18	WideResNet50
Extracted Layers	2 and 3	2 and 3
Feature Vector Length	384 (128+256)	6144 (512+1024)×2×2
Subsequent Processing	Select randomly vector of length **50**	Adaptive avg. pooling, Coreset subsampling by **0.5%**
Patch Grid Size	256×398	255×397
Image-wise Anomaly Score	Maximum value of patch grid (Mahalanobis distance to the modeled Gaussian)	Maximum value of patch grid (ℓ_2 distance to the nearest neighbor)

For PatchCore, other selected hyperparameters include *target_embeded_dimension* as 256(the second adaptive average pooling target), *anomaly_scorer_num_nn* as 1(the number of nearest neighbor to compare with), *patchsize* as 2 which is like the kernel size in CNN model, but unfolds the values instead of doing weighted sum. This brings advantage over PaDim because the neighboring vectors are also extracted and further evaluated, which is why PatchCore's feature vector length is multiplied by 2×2. For both

models we chose to extract layer2 and layer3, because layer1 of the model focuses on superficial features, while later layers focus on deeper features[29]. Layer2 and layer3 have different channel lengths from different backbones, which in the ResNet-18 case is 128 and 256 and in the WideResNet50 case is 512 and 1024.

Both methods are ”trained” with batch size as 1. Unlike methods that train with images with good and bad labels, unsupervised methods are conservative in nature because only good images are involved in training. The line between good and bad images is never drawn, so there is no over-fitting in a machine learning sense. What we have is the knowledge of the good images, leveraged to differentiate anything that differs. In this sense we need to have as many distinct variations as possible, to be inclusive of all changes in passed samples. Therefore using 1 as batch size extracts perfectly individual sample's features, which serves the purpose of recording unique features in the memory bank(PatchCore) and in Gaussian modelling(PaDim).

All of the method evaluations were conducted using a RTX4090 gpu(24GB RAM) and i9-14900KF cpu with 64GB of RAMs.

4.3 Heatmap Normalization and Processing

By aforementioned methods, each image yields a patch grid that marks the patch-wise distance to the good images. After rescaling and normalization a heatmap image can be generated, with its heat indicating where it is most likely anomalous. The patch grids before rescaling, however, are not good for visualizations. Large portions of the patches are quite close to the normal image because anomalies only occupy a small region of the full image. Therefore most pixels in the patch grid are below 10. The anomalous patches which have higher intensities were not very high within [0,255], maximum of which is 62 in the worst case scenario. So they are not very much visible. An example of 3 different images' patch grids can be seen in Figure 26.

Figure 26: 3 patch grids from a perfect image, a good (flawed but non-anomalous) image, and a severely anomalous image.

If viewed closely, on the rightmost image you can see the squiggle shaped anomaly on the black background, which has the highest intensity value(47). Histograms of the three patch grids are shown in Figure 27.

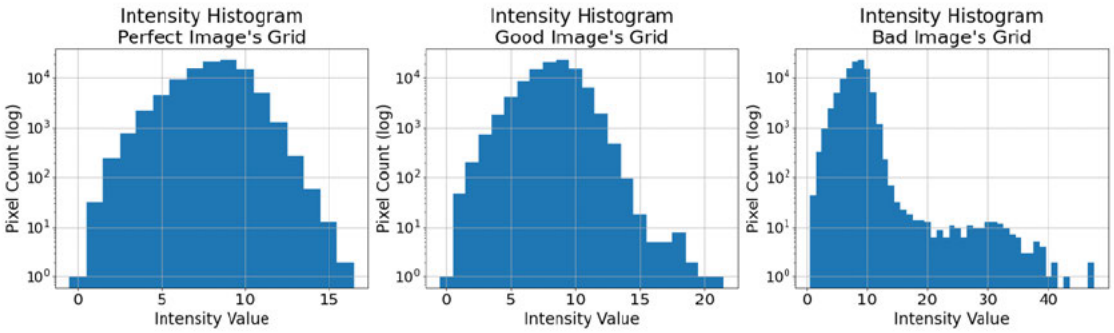


Figure 27: Histograms of the three patch grids.

In the perfect image's case, the intensities are fairly low, due to its resemblance of the passed chips in the training set. And the more the chip deviates from the perfect chip the bigger the distance registered in the patch grids. The center histogram is overall shifted to the right in comparison. In the rightmost histogram of the worst chip we can see a much larger overall intensity. The largest values on the x-axis are presumably the squiggle pixels.

In the unsupervised anomaly detection field, it is common to normalize by min-max normalization. Each image is normalized by subtracting the minimum value of the image and dividing by the range between the maximum and minimum value. Here you can see from the Figure 28 that the perfect image's intensity becomes higher because its values are stretched from [0,21] to [0,255] in the same way how the bad image is stretched from[0,47] to [0,255]. This way the highest value within each image would be emphasized, but it loses a fair judgment across different images.



Figure 28: Min-max normalization of the patch grids.

On the background you can also see that there is patterned noise in the shape of the channels in all three images. We have tried to normalize with different methods on these images, including: Z-score Normalization (subtract the mean and divide by the standard deviation), Robust Normalization (subtract the median and divide by quartiles), Percentile Clipping (remove the top and bottom 1% and then do min-max). The results are shown in Figure 29. The first 3 different normalizations are basically the same, and the last percentile clipping looks worse. In the grids the maximum pixels are the anomalous pixels, clipping those would lead to a blurred effect, adding emphasize on

the medium valued pixels.

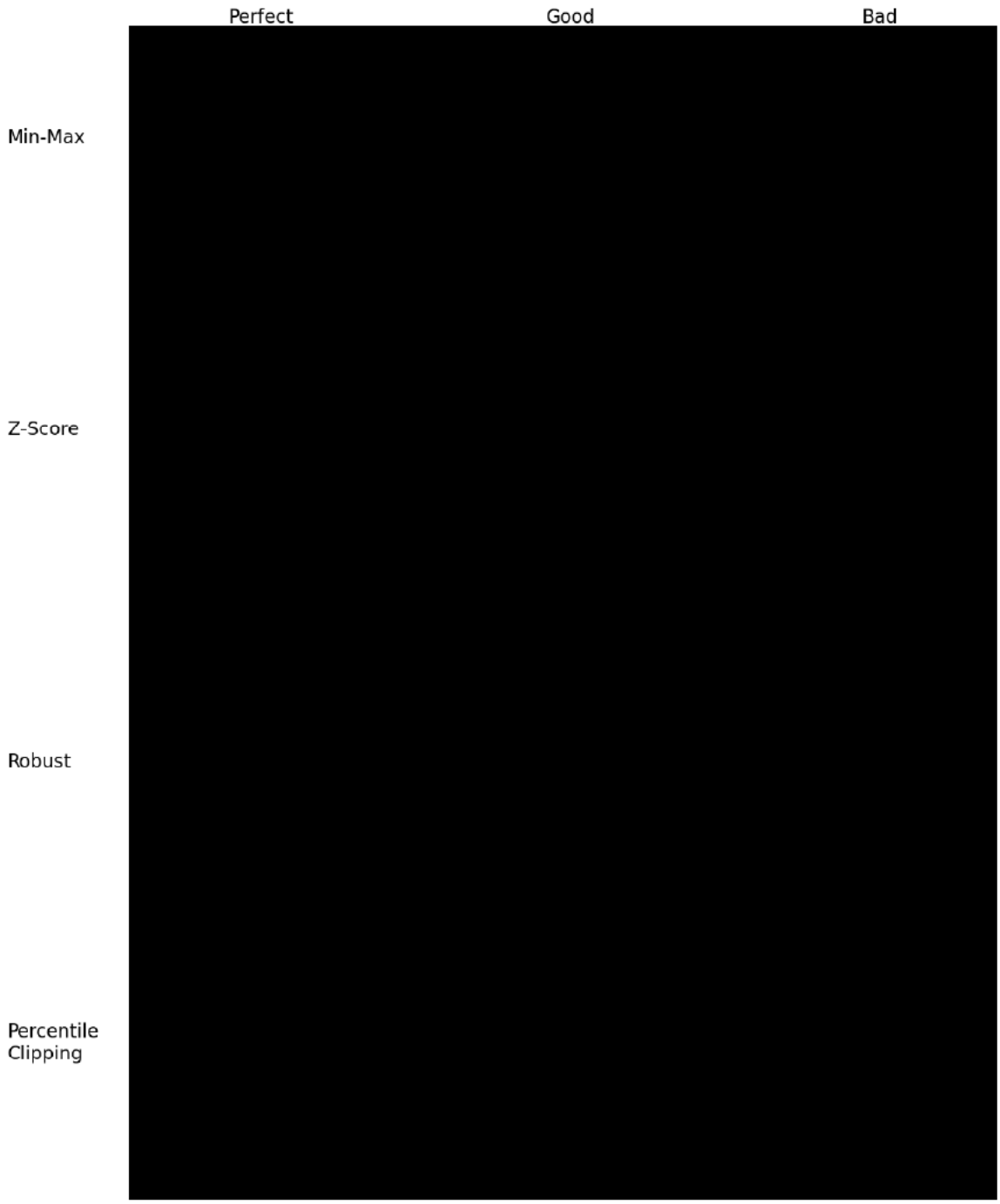


Figure 29: Normalization variants across datasets.

Consequently, the best normalization for our dataset is the global min-max normalization. Normalizing the heatmaps by the minimum and maximum value of all images. This way the anomalies are evaluated consistently across the images and the distances are stretched equally. The global min-max normalized images are shown in Figure 30. For this method an extra knowledge of the maximum value across all the patch grids is needed, but it is unavailable when inferring each image. Empirically the maximum value is 50 when the patch is a worst anomaly.



Figure 30: Global min-max normalization of the three grids

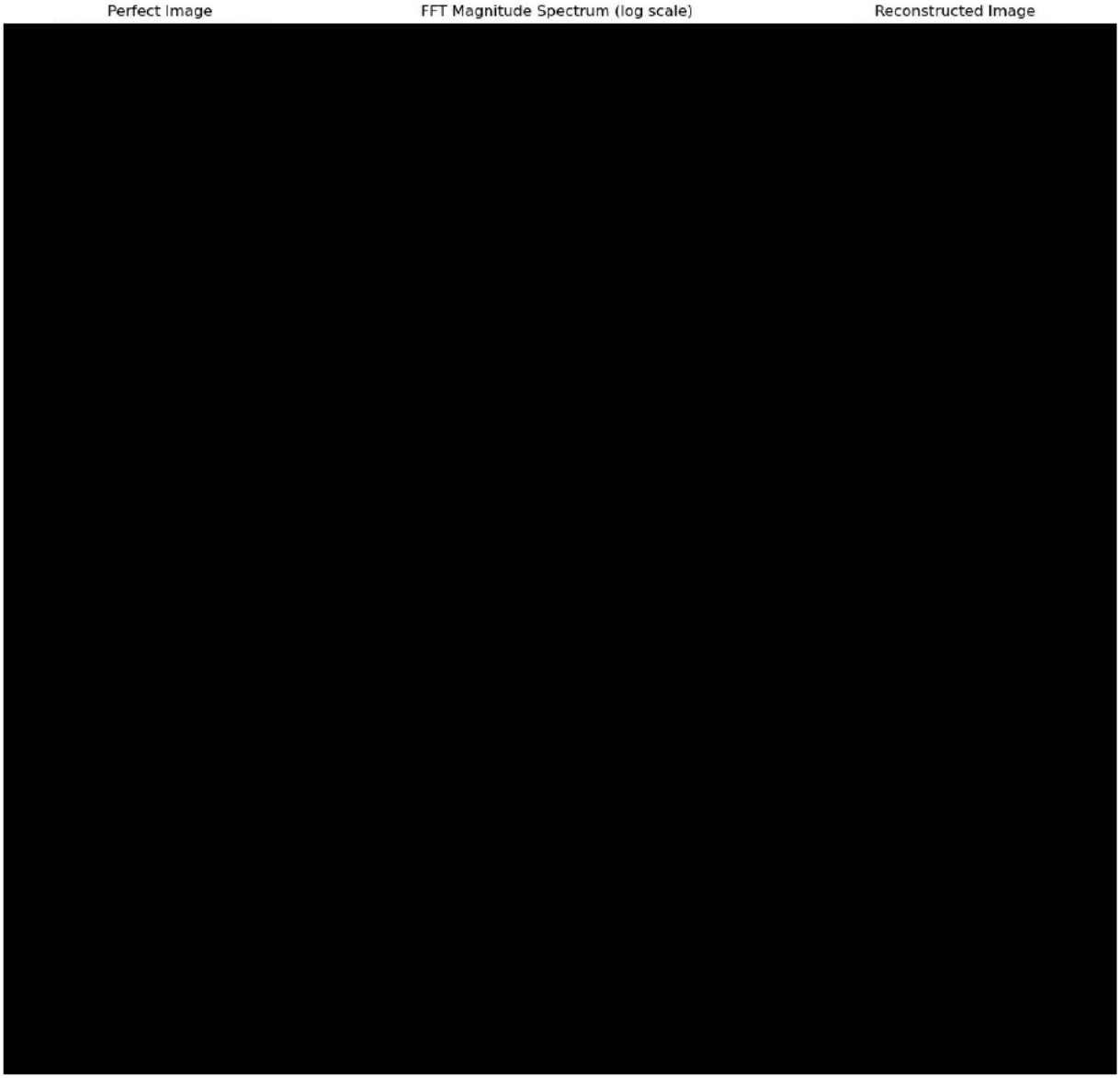


Figure 31: Reconstruction process using fast Fourier transform.

To remove the background noise fast Fourier transform is applied. In Fourier domain

exist a number of thin bright stripes across the spectrum. Removing those and reconstructing the image generates the denoised heatmap, see Figure 31. Nothing is left in the perfect images' reconstruction, and only a tiny bit of red remains in the center right of the good image. In the worst image the anomaly is fully preserved against a clean background.

To better visualize the remaining pixels in the heatmap, we have chosen the 'hot' color map (discussion on other color maps can be seen in Appendix A.3), overlay the original image by a factor of 0.5. Results of the good and bad images are shown in Figure 33. In the first row the top left corner and center right part exist slightly red marks at the position of the artifacts. The lower two images show the effectiveness of the model in detecting severe anomalies. The greatest emphasize is placed where the anomaly goes across channels, and reasonable heat is marked on defects of lower severity.



(a) Good image original. (b) Good original overlayed with heatmap.



(c) Bad image original. (d) Bad original overlayed with heatmap.

Figure 33: Comparison between the original chip and overlayed images (the good image and the bad image).

5 Results and Discussion

In this section we start by comparing the PaDim and PatchCore's efficiencies and qualitative performances. Then go deeper into quantitative aspects of model performances.

Based on the observed results, we present suggestions for rule based sorting of the chips.

5.1 Training and Testing Efficiencies

Table 3: Efficiency performance comparison.

	PatchCore	PaDim
Training Set	50 images	100 images
Training Time	5.5 min	30 s
Model Size	24.7 MB	998 MB
Training RAM Usage (Peak)	24 GB	64 GB
Training CUDA RAM Usage (Peak)	16 GB	7 GB
Testing Time Per Image	7s	1.14s

As shown in the Table 3, PatchCore has longer training time but less RAM pressure. While PaDim has a faster training time but heavier memory usage. This is because PaDim's code implemented feature vector concatenation in full images instead of patch-wise in PatchCore.

The saved models, for PaDim which is the mean and the covariance of each patch's Gaussian, ended up to be 998MB. Much larger than the subsampled and saved patch features in PatchCore(24.7MB).

Testing time turns out to be very different. PaDim takes only 1.14 seconds per image but PatchCore takes 7 seconds. A FAISS[30] vector search turns out to be more time consuming than Mahalanobis distances calculation(PaDim).

5.2 Qualitative Comparisons Between PatchCore and PaDim

Early experiments using the Anomalib[31] library as well as other publications[32] state that PaDim has an advantage in inference speed as well as sensitivity. The same shows that PatchCore is more robust but a bit slower. The two models differ by only a few percent in performance. But all of the published datasets were resized into 224*224 and well annotated.

Working on our dataset, the methods show much larger difference. PaDim is more sensitive like claimed. It gives a lot more alarms than PatchCore as shown below in Figure 34. On the upper left we can see that PaDim gives a lot more alarms on the "basis" part of the chip, which is uninteresting for anomaly detection. While on the right PatchCore gives less intense but more differentiated markings on cuts across channels. In the lower image PaDim is subject to the inner circle drifting, and gives alarms all over the inner gray circles. While in comparison the PatchCore model better combats this variation. Since patch features are modeled as multi-variate Gaussians in PaDim, the method is expected to be more sensitive than the pairwise comparison mechanism

used in PatchCore.

Unlike PaDim, the PatchCore heatmaps output is very stable with only mild noise in the background. The overlayed images show that the focus of the heatmap lies in the white channel region, which is a great improvement compared with PaDim.



(a) PaDim output of image 1.

(b) PatchCore output of image 1.



(c) PaDim output of image 2. (d) PatchCore output of image 2.

Figure 34: Heatmap comparison between PaDim and PatchCore.

5.3 Anomaly Score Statistical Evaluations

As mentioned in Table 2, each testing image is assigned an anomaly score which is the maximum value from the patch grid. The statistical distributions of the anomaly score by class from the 2 models are illustrated in Figure 35. Where the PaDim model hardly separates the good images from the bad images, but PatchCore does much better separation with difference in mean values and the overall shape of the distribution.

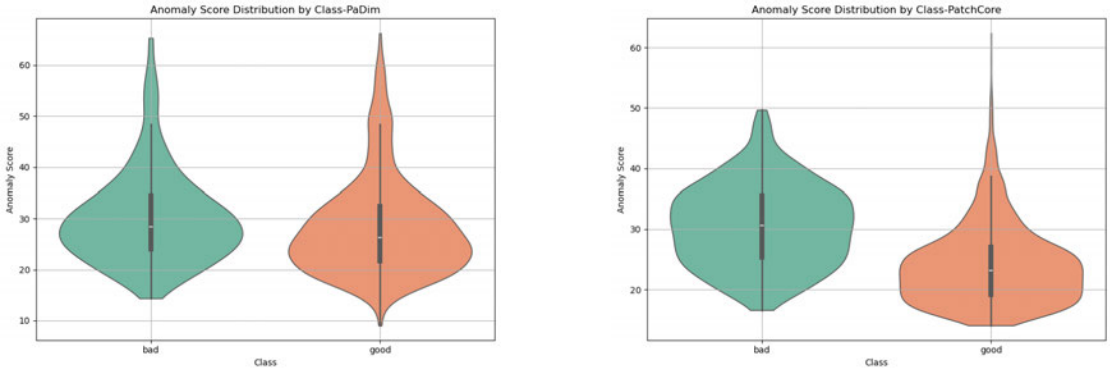


Figure 35: Anomaly score distributions from the 2 models.

Being far from ideal, a fraction of the good chips can still be sorted out using a threshold

on the anomaly score, see Section 5.5.2. Which correspond to the bottom of the ”vase” shape distribution on the right of Figure 35. These are chips with anomaly scores lower than all of the bad chips, unfortunately it only takes up 13% of all good chips in the PatchCore case.

The false alarms from the good chips have contributed to the high values in this category, and the subtle defects from the bad chips have lowered the model's alarm. The issue lies in the nature of the anomalies, where many visible differences are insignificant, but some small differences are critical. This is why the anomaly score is not sufficient for anomaly classifications, therefore in the following Section 5.4 we elaborate the cause of the false alarms as well as the sorting challenges. Section 5.5 explains the development of our sorting rules.

An AUCROC curve can be drawn using the labels and the anomaly scores, see Figure 36, where the area under curve for PatchCore is 0.7691 and for PaDim is 0.5734. Clearly PaDim doesn't differentiate between good and bad chips, whereas PatchCore has reasonable well differentations. Therefore the following sections exclude PaDim for evaluations.

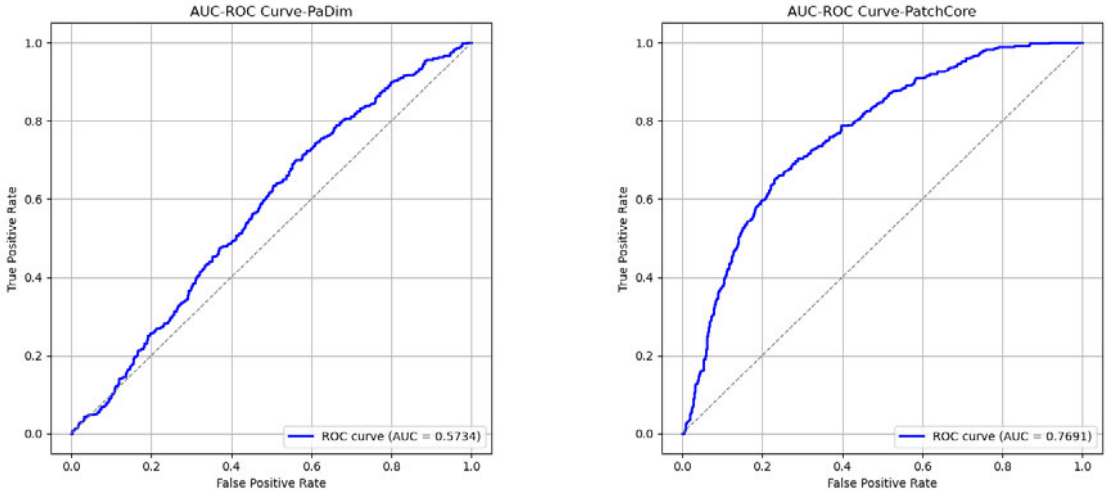


Figure 36: AUC-ROC curve of the 2 models.

Another reason why the anomaly score is not useful is because the highest scores across all images are from patches with pixel errors. The top three anomaly scores: 62, 57, 54 are all results of pixel errors. The three corresponding locations are shown in Figure 37. Given that these parts exhibit large gradients, it is understandable that the model places the greatest emphasis on them. Beside these extreme causes of false alarms, a few others that trigger the model response or complicate anomaly classification are elaborated in the following Section 5.4.

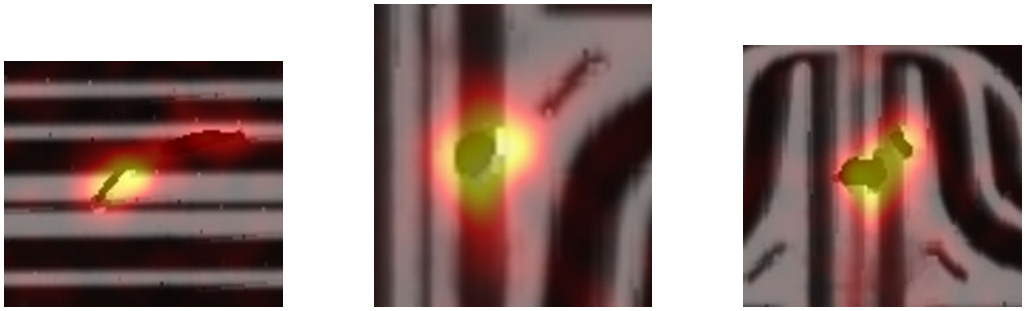


Figure 37: Top 3 patches with the highest anomaly scores (across all testing images).

5.4 Challenges in Differentiating Anomalies

The model has done its work by marking out where it sees a difference, and a heatmap is generated correspondingly. The question is, how can we utilize these heatmaps? Is the heat differentiable enough for all the anomalies? Can we simply discard the chip when there is a heated region? The answer is negative.

In this section we will list out the major interference that triggers the model's alarm. Also listed are the intriguing borderline cases of anomaly classifications to show the complications in chip productions and the subtle anomaly standards.

5.4.1 Major Causes of False Alarms

Besides the pixel errors from FocalSpec, which leads to false alarms, other processes during productions are also liable for causing harmless variations on the chips. Lots of these variations have huge visible differences against normal chips, but are not anomalies.

HIP Defect A very common defect around the edge of the chips that comes in large shapes and numbers (accounts for a third of the false positives for the channel sorting rule). Example of which is shown in Figure 38.

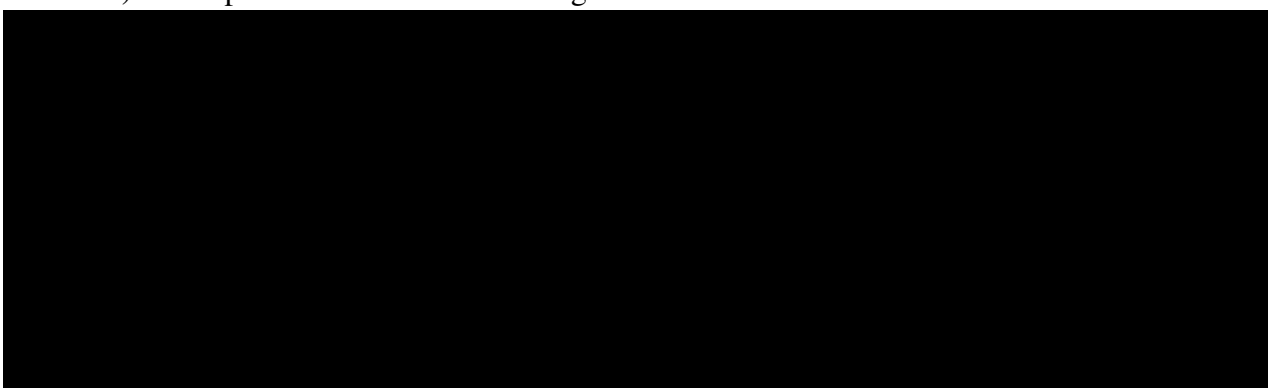


Figure 38: HIP defect example with model output.

Shim Defect Defect of a production part, which end up leaving visible differences on the chip. Example in Figure 39. Not harmful and takes up 25% of the false positives in channel sorting.



Figure 39: Example of Shim defect and model output.

Non-blocking Debris Debris when small is OK to be connected to the thin channels or in the round wells. Small debris widely exist in many chips, some trigger little heat that is not sorted but visible, for example Figure 40. However, when the debris is bigger and ”wrongly” placed, could be harmful like (g) and (e) in Figure 55. Which block the flow of the fluids.



Figure 40: Small debris example with model output.

Pixel Errors This type of false alarm is deeply rooted in the initial imaging of the point cloud, where the disturbance in the 3D imaging has ended up leaving holes after removal. The end result is having blocks of missing pixels in 2D images, which is heavily emphasized by the model. Examples of which is shown in Figure 41.

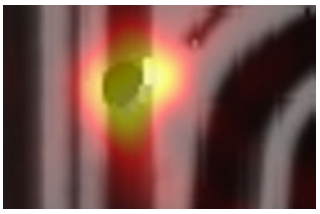


Figure 41: Example of pixel error caused by optical interference (reflection or refraction).

Injection Moulding Variations caused by production process, which leaves dark spots on the chip channels. Injection moulding commonly appear as light gray to dark textures by the edge of the thin channels, which is not alarmed by the model. An intense and alarmed case can be seen in Figure 42.

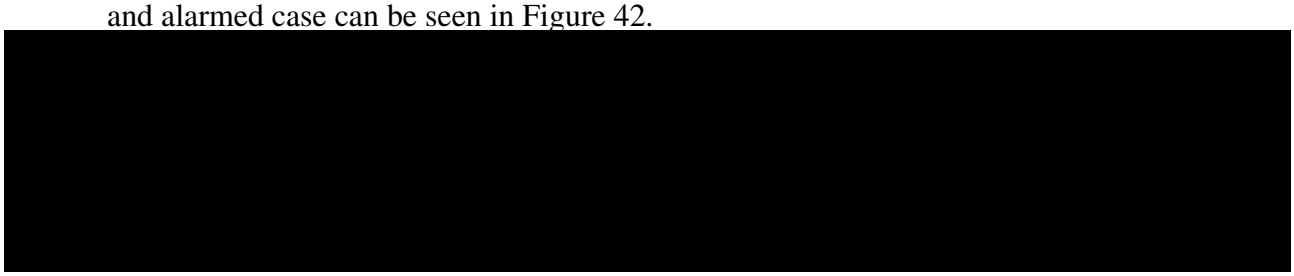


Figure 42: Injection moulding example and model output(3424-6-b-1).

5.4.2 Difficult Anomaly Border Cases

Production variations have posed many challenges when the chip has tiny defects or acceptable variations. Some anomalies are so small and barely visible; some are actually anomalies but passed because they are not in critical positions. Examples of those cases are listed below.

Small Debond The PatchCore model is sensitive enough to locate small changes, but sometimes it is hard to sort it out as it is so close to very minor differences. For example in Figure 43, though the model highlights on the tiny debond, it is hard to localize it as an anomaly. This small piece is mildly highlighted and off channel, which can not be sorted out. This case applies to the ①,⑤,⑦ of the Table 6 from the rule to sort good chips. Its significance contradicts with its size, thus the model is not highlighting it enough. While some other widely existing and harmless artifacts are more emphasized by the model.



(a) Chip with small debond (b) Model output

Figure 43: Small debond in chip and model output.

50% Rule In productions some anomalies were purposefully accepted, because they were not in critical positions or they cause no harm to their neighboring channels. The debris and debond in the productions were allowed when they do not extend over 50% towards its neighboring channels. An example of which is shown in Figure 44, where

the anomaly is highlighted, but there is no applicable rule to measure how far it is to its neighboring channels. Confusingly there is another very similar debris that does block the channel as shown in Figure 45a. Differentiation of which would be highly challenging. Another extreme pass case of this rule can be seen from Figure 45b, where the anomaly is extended just 50% in between. A subtle fail from this rule is the ⑨ of Table 6. This rule contributes to 25% of the false alarms in channel sorting.

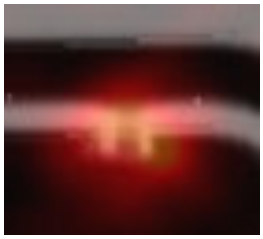
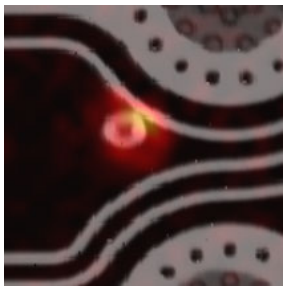


Figure 44: A passed Debond (within 50%) example with the model output.

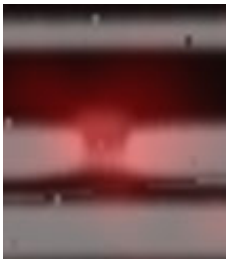


(a) Blocking debris example of a failed chip.

(b) A 50% borderline case of a passed chip.

Figure 45: An example of anomaly debris and a borderline pass example.

Half-blocking Debris This is a particular case when a debris dwell in the back channel, but only blocks half of it. In this case the chip is fine as the fluid can still flow through. These debris are also highlighted by the model, but are not harmful. Example of which is shown in Figure 46. The difference on the channels are bound to be detected by the model, but to mitigate or reduce the model's response on these flaws is not possible, as these are visibly different from normal chips. Using algorithms to differentiate those would also be challenging, proper measurement of the deformation and the width of the back channel requires much effort. When this debris grows a bit longer in the case ⑨ of Table 6, it becomes a valid anomaly that violates the 50% rule.



(a) Half blocking original.

(b) Model output.

Figure 46: Half blocking example with model output.

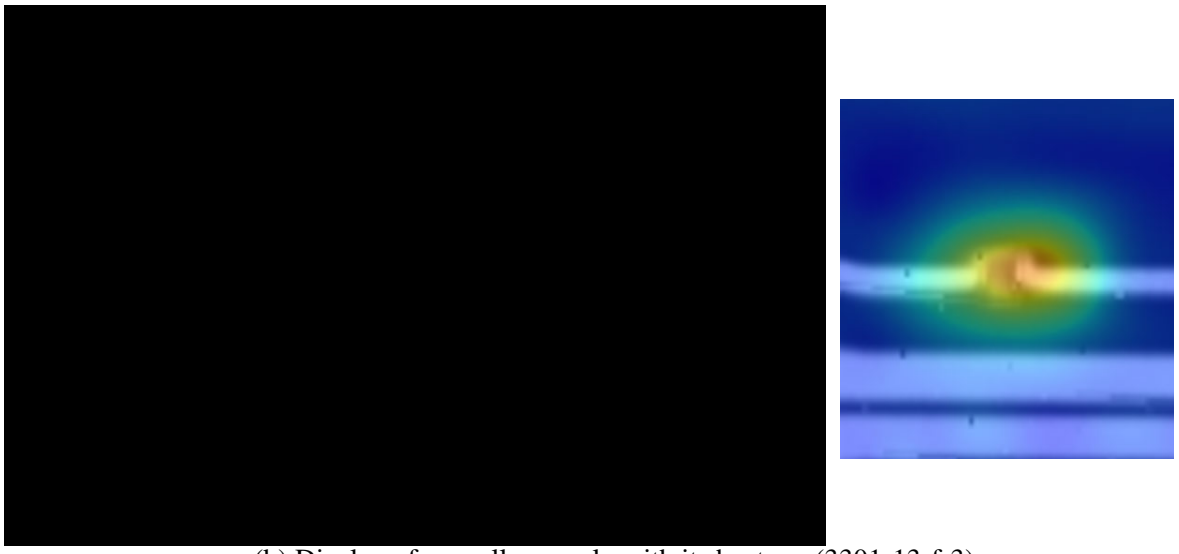
5.5 Rule Based Sorting — PatchCore

5.5.1 Rule to Sort Out Bad Chips

Under different anomalies the heatmap output differs in size and intensity, which helps to do further anomaly classifications. Two major types of anomaly examples are shown below in Figure 47. For image classification, the first type can be classified by measuring the largest connected component size. The second type would need to mask the heatmap by the original channels, where further thresholding is applied to sort out the anomalies.



(a) Display of a long moderate anomaly with its heatmap (3427-6-f-4).



(b) Display of a small anomaly with its heatmap(3301-13-f-3).

Figure 47: Display of 2 major heatmap responses.

Large Anomaly Rule First exclude the leftmost and rightmost 20% of the images to avoid response from the large HIP defects. Then detect connected components with size over 2000. Under different size thresholds the true positives and false positives show a tendency of decline, as shown in Figure 48. Where the green line is the number of bad

chips sorted out from this rule and the red line is the number of falsely rejected chips.

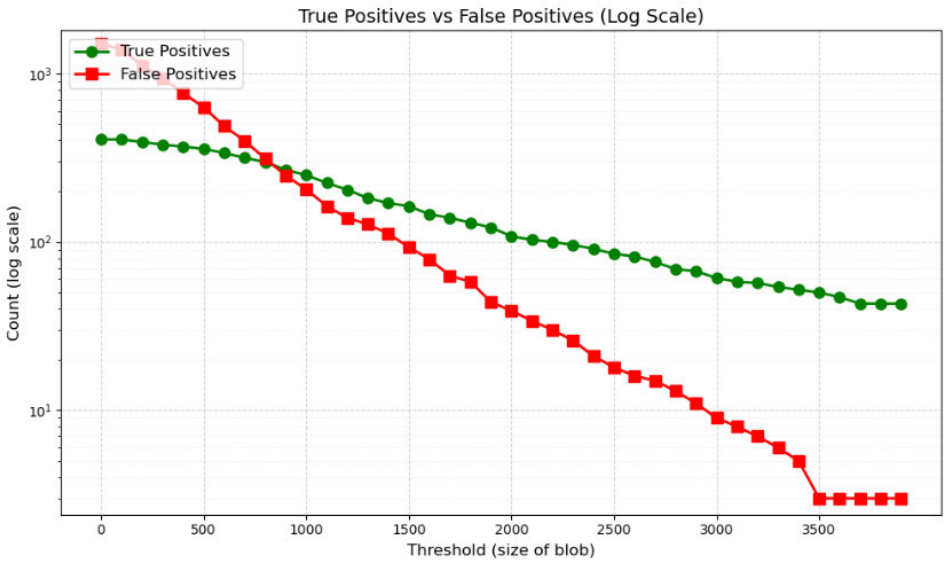


Figure 48: True positives and false positives vs thresholds.

We have chosen two thresholds and verified the sorting performances as shown in Table 4. Most of the falsely rejected chips are the HIP defects at approximately the same position. Under large blob size threshold we can avoid selecting these mid sized HIP defects from the heatmap, but in the meantime losing quite a few bad chips. The anomaly with number ① is a defect within the 50% criteria by the production manuals, and the number ⑥ is caused by removed debris' original location. As mentioned in Section 4.1.3, the debris removal will leave a hole on the chip plane when seen from above. This will leave a debris silhouette in 2D as 0 value pixels, which triggers model response.

Table 4: Large anomaly sorting performance. Each FP chip is shown per row with centered indication image.

Threshold Value (Blob Size)	True Positive	False Positive	FP Chips Indications
3000	61	9	① 3425-5-b-4
			② 3299-4-d-2
			③ 3299-7-b-1
			④ 3299-9-b-1
			⑤ 3299-10-b-3
			⑥ 3299-10-g-2
			⑦ 3302-4-f-2
			⑧ 3302-6-f-4
			⑨ 3302-9-a-3
4000	41	2	② 3299-4-d-2
			⑨ 3302-9-a-3

Channel Anomaly Rule The heatmaps masked by the corresponding image's channels show where it is likely to be anomalous on the channels. Taking different values as thresholds from the masked channel image we segment the heated regions. When this

heated region is larger than 250 pixels and the corresponding original region's intensity is larger than 125(to sort out greyish HIP defect regions). Applied with an intensity threshold we are able to segment the channel anomaly regions and hence fail the chip if such spots exist.

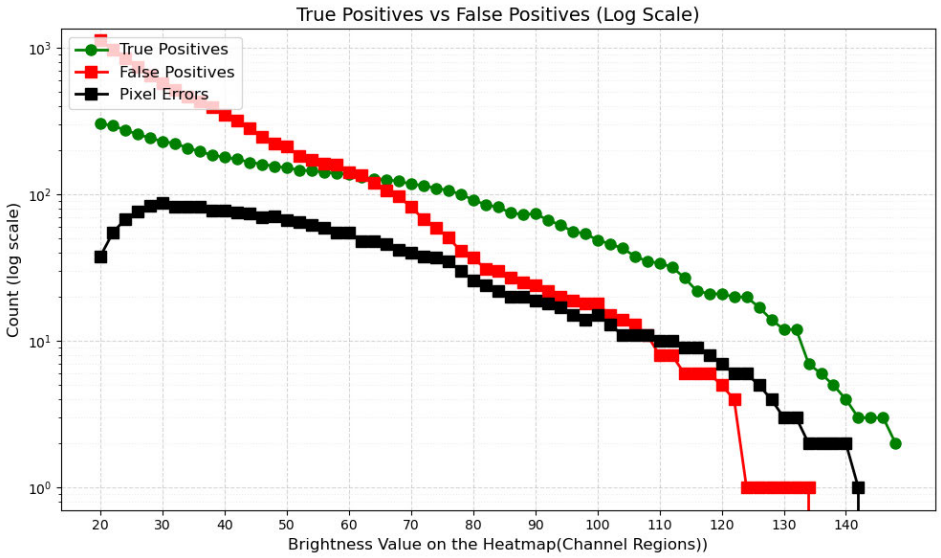


Figure 49: Heatmap channel bright spot rule performance.

From Figure 49 we can see that the false positives coexist with true positives. A large number of bright spots on the channels are also the result of the pixel errors, which creates confusion for the model when it sees a 0 intensity pixel among the white channel region of intensity 200. Increasing the threshold still keeps quite a number of false positives. This is due to the complicated failing standard that actually allowed anomalies in uncritical positions, which still triggers the model because of its visual differences.

Table 5: Channel bright spot sorting result.

Threshold Value (Brightness)	True Positive	False Positive	False Positive Cases	Indications
50	182	220	57 non-blocking debond 16 non-blocking debris 51 Shim defects 71 HIP defect 7 bad chips(True Positives) 5 injection moulding 2 pre-processing error 2 pixel errors(non-zero)	
90	74	24	16 non-blocking debond 3 non-blocking debris 1 HIP defect 4 bad chips	

From Table 5, most of the false positives are triggered by HIP defect, Shim defect, or non-blocking flaws. It happens that there are two chips being chopped off in preprocessing and caused alarms by the edges, and another two chips having visible pixel errors but did not get sorted out using 0 values. Details of the seven bad chips will be elaborated in the following case analysis chapter.

5.5.2 Rule to Sort Out Good Chips

As mentioned in Table 2, the anomaly score of each image is the largest value in the patch grid. This value is the largest patch-wise distance to the normal image, indicating the most extreme difference of the whole image. By select images with a lower anomaly score, we can sort out what the model believes to be the better images. Example of a perfect chip's heatmap is shown in 50, where no distinct bright spots can be seen. Its anomaly score is very low (14.05).



Figure 50: Example heatmap from a flawless chip.

Sorting results under different thresholds can be seen from Figure 51 where the green line is the number of good chips sorted from this rule and the red line is the number of bad chips that slipped in.

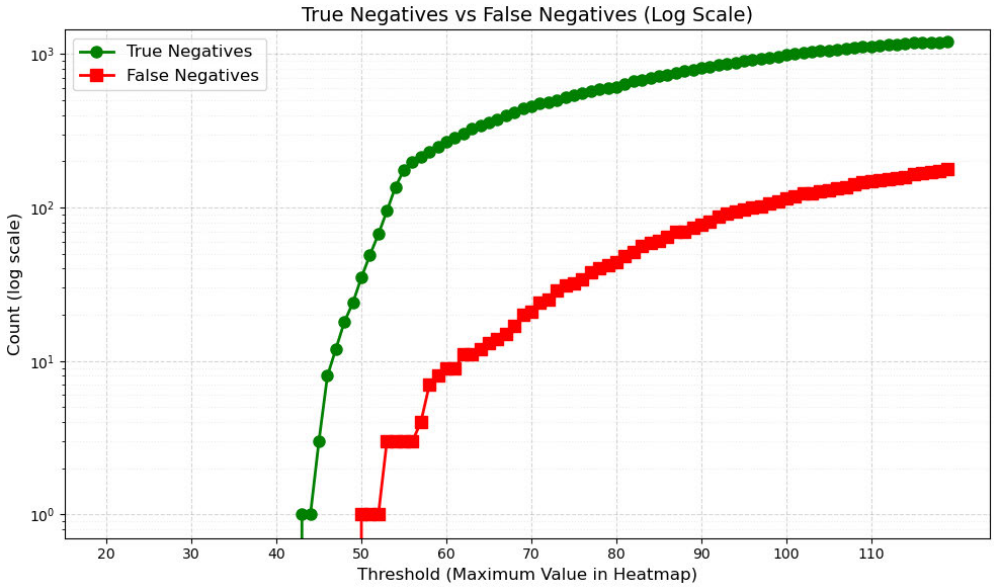


Figure 51: True negative and false negatives by different anomaly score thresholds.

Under different thresholds there is a tradeoff between the number of good chips and the number of wrongly included bad chips. Table 6 displays a series of mistaken chips. Case ① is a tiny debond from the channels, which extends a little bit beyond 50% towards the neighboring channel. The same is ⑤ and ⑦. Those three were marked by the model with very little heat, details will be further discussed in the following case analysis part. The ⑨ case is a debris that blocked half of the back channel but extended a bit beyond 50% towards the neighboring channel. The ⑧ are two very small scratches

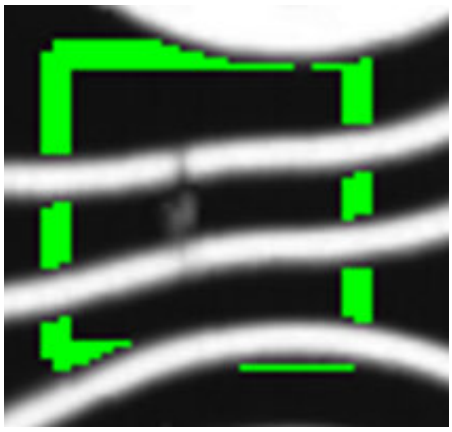
on the chip. Case ⑥ is an interesting borderline case which can be considered a pass, but it is easily confused with injection moulding or debris, one operator stated it could be a Shim defect which is not harmful. ④ is a current system automatic fail which turns out to be OK. The remaining case ② and ③ are two chips that our operator found no issue at all. These are likely to be cases when a chip falls on the ground or are lost during productions, ending up failed in FocalSpec.

The higher the heatmap maximum value, the more likely it is to be a bad chip from the model's perspective. From Table 6, when we take a conservative value of 49, where the model sees very little difference from the good chips, we yield only 24 good chips with no bad chip at all. However, when taken 56, we gain 197 good chips with only 3 bad chips(false negatives). These 3 marks the initial mistakes the rule makes. These are the first 3 bad chips that our rule has risked into the group of good chips. It is surprising that only 1 of the 3 is a valid anomaly, only ⑦. The other 2(② and ④) are fine as shown in Table 6.

Table 6: Rule sorting result under different thresholds. Each FN chip is shown per row with operator error indication image.

Threshold Value (Anomaly Score)	True Negatives	False Negatives	FN Chips & Operator Indications
60	269	9	① 3301-21-b-2
			② 3301-23-a-1 Controversial
			③ 3301-23-a-4 Controversial
			④ 3426-5-c-4  System fail mistake (magenta fail)
			⑤ 3426-6-b-4
			⑥ 3427-1-c-2
			⑦ 3299-7-a-3
			⑧ 3299-9-f-2
			⑨ 3302-11-b-3
56	197	3	② 3301-23-a-1 Controversial
			④ 3426-5-c-4  System fail mistake (magenta fail)
			⑦ 3299-7-a-3
49	24	0	None N/A

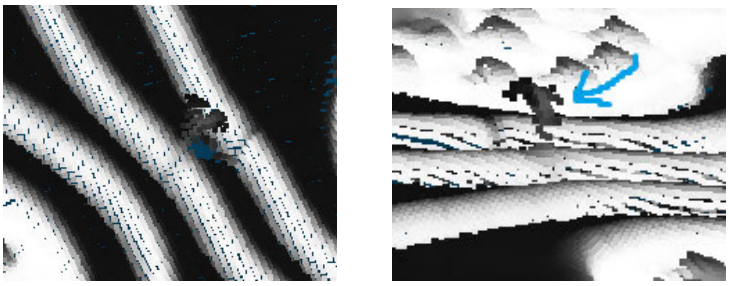
Even this one valid anomaly reveals something deep in the imaging and anomaly classifications. Table 6 case ⑦ shows nothing but a hole of a pre-existing debris. We remind the reader that a hole in the 2D image meaning this debris is above the chip plane Section 4.1.3 . In 3D denosing, RANSAC would select and remove those as pixel errors, and this is why our preprocessed image looks different than the one currently in operator view. Difference caused by our denoising is shown in Figure 52, where the preprocessed image on the left omits the debris removed in 3D.



(a) 2D view after preprocessing. (b) 2D view in current operator view.

Figure 52: 2D view difference caused by 3D denoising.

In this sense this debris anomaly is OK to pass, because: 1. it is above the chip structure, 2. it will be removed by the operators when they tape it. However, it is not guaranteed to be a debris, as there is a possibility of optical interference which forms artifacts like these. Further checking in 3D is displayed in Figure 53. The piece of debris is slightly detached from the chips between the channels. RANSAC removal of the mentioned debris is shown in Figure 54.



(a) Debris top view. (b) Side view of the debris. (c) Side view2 of the debris.

Figure 53: 3D view of the floating debris in 3299-7-a-3.

Figure 54: RANSAC outlier selection of the mentioned debris.

Despite its unusual appearance in 3D imaging, our production team confirmed that it is an anomaly. Though it appears to be a floating debris outside the chip, it is causing malfunctions in the chip. Which means that it is an imaging problem in the FocalSpec, likely the result of optical interference, somehow shifted the debris.

If the threshold is increased to 60, as shown in Table 6, the number of false negatives grows from 3 to 9. These new thresholded chips are very worthy of looking into, as those chips reveal the line between a good and a bad chip by the model's anomaly scores. What kind of defect does the model tolerate and give a low, undifferentiated anomaly score? Table 6 shows that those mistakes are indeed very subtle anomalies. This also tells us that the best possible threshold is within these numbers(56-60).

5.5.3 Sorting Summary

With carefully picked thresholds, we are able to summarize the performances of the three rules, rule sorting results are shown in Table 7. In summary we have picked out 159 bad chips(the whole of 406), and 197 good chips(out of 1498), with the cost of keeping 3 bad chips and falsely reject 42 good chips. Because we are doing the classifications rule-wise, a false negative rate and false positive rate would mix between the rules, which is not a good indicator for this project. Hence the 1.5% mistakes from the good chip rule and 20.89% mistakes from the bad chip rules should all be called error rates.

Overall 401 chips are sorted, taking up 21% of all the 1904 chips, and the accuracy of our sorting on this subset of chips is: 88.78%. The selected threshold for the large anomaly rule is 2500, for the channel anomaly rule is 90, for the good chip rule is 56.

Table 7: Rule sorting summary.

Rule of	True Positives	False Positives	True Negatives	False Negatives
Large Anomaly	85	18	0	0
Channel Bright Spot	74	24	0	0
Low Anomaly Score	0	0	197	3
Summary	159	42	197	3

The confusion matrix of the sorted chips is:

$$\begin{bmatrix} \text{True Positive (TP)} & \text{False Positive (FP)} \\ \text{False Negative (FN)} & \text{True Negative (TN)} \end{bmatrix} = \begin{bmatrix} 159 & 42 \\ 3 & 197 \end{bmatrix}$$

The chip production yield is around 80%, meaning 1 out of 5 chip is discarded. This 20% discard rate can be used as a standard for rule sorting false positives. Say we secure 5 bad chips and 1 out of the five is a good chip, this is still quite productive as we are having false positive rate of 20%, which results in falsely rejecting 5% of the full production if all the 20% bad chips are detected by this rate. We believe this is a good rate for the sorting standard of the bad chips, and our error rate for the bad chip rules is

20.89%$(42 \div 201)$ in total.

The most critical mistake our sorting makes is the false negatives. Consider that each chip works as a diagnostic tool for patients, an anomalous chip might end up indicating the patients' status wrongly and leads to mistaken analysis, even prescriptions. These mistakes are the least tolerable of the 2(discarding good chips vs keeping anomalous chips). A tight threshold is chosen for this purpose, leaving only 3 false negative chips. The error rate for the good chip rule is 1.5%$(3 \div 200)$.

5.5.4 Contradictions with Annotation

The model visualizations have also helped us to locate quite a few overlooked anomalies. The 7 bad chips in the channel sorting rule along with other 2 are displayed in



(e) 3427-5-c-1 (f) 3299-3-d-3 (g) 3302-3-c-3 (h) 3425-1-g-4



(i) 3301-12-e-2

Figure 55: False negatives by annotation.

By the low maximum value rule, we have also found 3 controversial failed chips, namely: 3301-23-a-1, 3301-23-a-4, 3301-27-d-1. Which can be cases of loss during production (fallen on the ground or lost).

5.5.5 Newly Found Deformation Marked by the Model

Interestingly a few ”new” types of anomalies are marked by the model. Sometimes visual differences are huge but not noticeable by human eyes. Examples of which are Figure 56 where the model noticed an overall warping of the channels and Figure 57 where a patch of the well is somewhat dark.

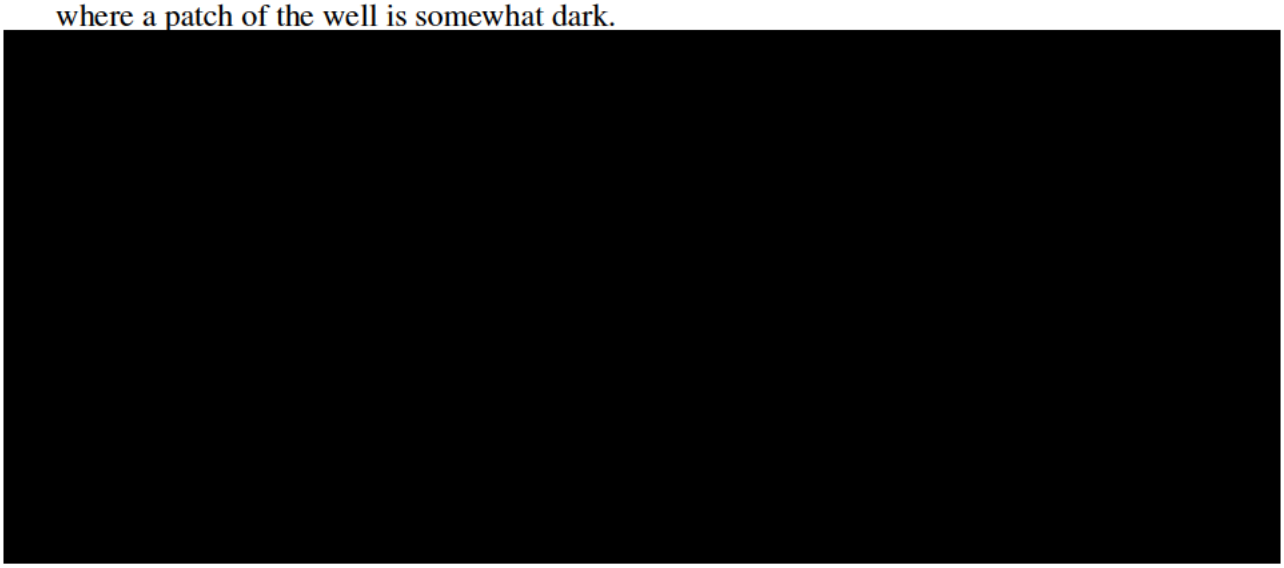


Figure 56: A chip(3427-4-a-4) warping marked by the model.



Figure 57: Grey patch on the well(3302-8-c-1) marked by the model.

The darker patch in the Figure 57 is very interesting. Checking the 3D point cloud data we see that this dark patch is the result of chip bending inward at that location, see Figure 58.



Figure 58: Dark patch location viewed in 3D (various angles).

Further, this chip has another subtle dark patch in another location, mildly marked by the channel. The model response and its corresponding 3D images are shown in Figure 59.



Figure 59: Dark patch location2 viewed in 3D (2 angles).

The cause of these grey patches is uncertain. We are unsure whether it is the result of production instability or a mere imaging distortion. According to our microfluidics team, there has been no incidents of leakage caused by this denting, and they would doubt any deform can occur at the positions. Therefore this gray patch is only an imaging issue. On the other hand, our production team cannot give an affirmative answer on the warping in Figure 56. However, in retrospect there has been no incident of leakage caused by such deformation, hence this distortion is tolerable.

5.6 Model Efficiency Experiments Under Reduced Dataset — PatchCore

To improve efficiency, we have also tried to segment the chips into 32 subsections and evaluate the performance compared with the default setting in Table 8 (bold fonted). The testing was performed in an early stage with 495 images.

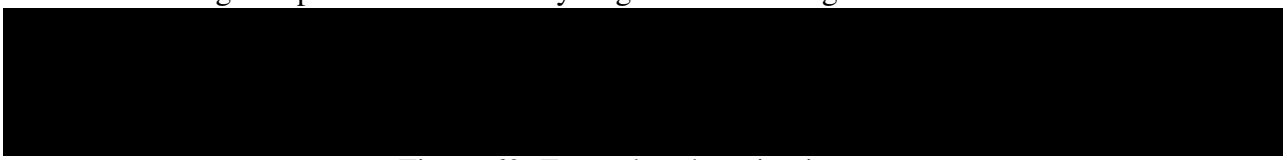


Figure 60: Example subsection image.

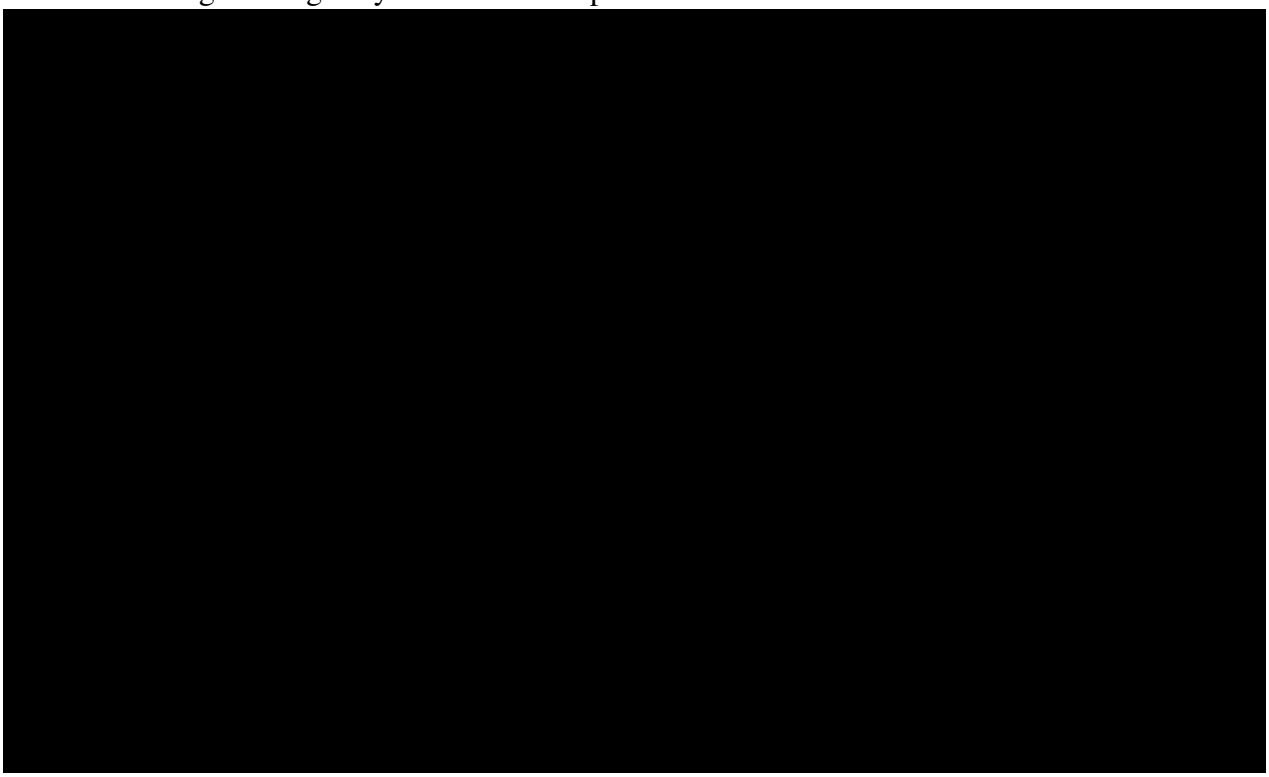
It is surprising that the smaller images yield no improvements in speed, as shown in Table 8. Smaller images would reduce computation individually but the overall time consumption remains similar to the full image settings. Reason could be that cropping the image makes no change to the Coreset, since it is the same percentage, one image or 32 images makes no difference. Which is why the model sizes stay the same. Consequently, the search time stays the same since the same number of patches are searched in a vector Coreset of similar size. The drastically increased I/O processing could also be the bottleneck, consider now there are 32 images to save instead of 1.

Table 8: Training and testing performance under various settings. The bold row indicates the default setting.

Data Format	Training Set Size	Training Time	Testing Time	Model Size (.faiss)
Full Images (2048*3178)	25	2 minutes	50 minutes	12.3 MB
Full Images (2048*3178)	**50**	**5.5 minutes**	**60 minutes**	**24.7 MB**
Cropped Images (1024*220)	50	6.5 minutes	62 minutes	26.7 MB
Cropped Images (1024*220)	100	21 minutes	90 minutes	53.5 MB
Full Images (2048*3178)	100	32+ hours	abort	–

The .faiss file size, which is essentially the memory bank extracted from the training set through Coreset sub-sampling mentioned in Section 4.2, increases in size linearly. We checked the performance under fewer training images than the default settings, as shown in the first row of Table 8. Though halved the memory bank size, the testing time only reduced by 16%. One could deduce that the image I/O consumes most of the testing time, or that the searching time in the memory bank is not linear with the number of vectors. The bright side of the phenomenon would be that a larger training set would still be efficient in training, if RAM accommodates.

By simple visual check, the heatmap prediction by the 25 training set model is reasonably well compared with the 50 training set one, see Figure 61. This suggests that increasing training set yields limited improvements.

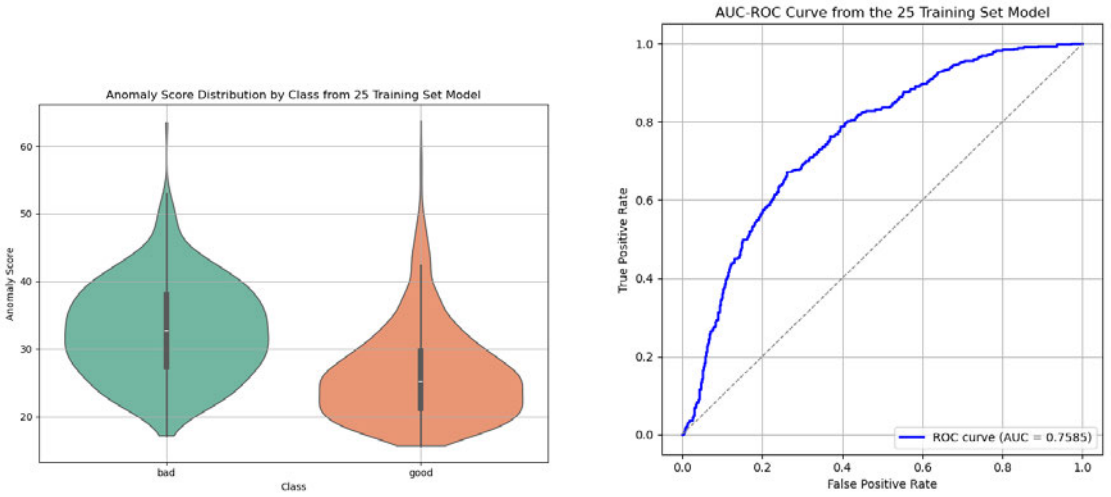


(a) Prediction by 25 training set model. (b) Prediction by 50 training set model.

Figure 61: Performance comparison, the 50 training set prediction has smoother background, and is more focused on the anomaly.

Further support for the 25 training set model is the anomaly score distribution and the AUCROC curve on the full testing set, see Figure 62. The area under curve is 0.7585, which is only slightly lower than the 50 training set AUC in Figure 36 (0.7691). The distribution shape is less as separated than the previous Figure 35, the new distribution's lowest value of the good class is 15.72 while the previous value is 14.05. Which is intuitively true as the previous model has more feature vectors to search with, possibly closer ones (the lowest closer by 1.67). We also have a lower value(16.59) in the bad class compared with previously(17.18). Consider now there are fewer vectors in the 25 training set, a lower distance to the bad image seems strange. Nevertheless, as the Coreset starts by randomly select 10 vectors, it is not guaranteed to have the same Coreset for each training which causes the minimum distance to differ slightly. The bell shape also shows that the anomaly score is less as concentrated in comparison, but the model performance is still on track, even with 25 images.

Why so? A major difference of PatchCore with PaDim is its comparison mechanism. The feature vectors get to compare with all vectors extracted from the image, even ones extracted from a different patch. Since we have a reoccurring structure in our image, one feature vector might has its nearest neighbor from another row of channels, which still makes sense. The duplicates in the chips combined with the location invariant vector comparison, helped PatchCore achieving better robustness and better anomaly differentiations.



(a) Distribution by class from 25 training set model.

(b) AUCROC curve by 25 training set anomaly score.

Figure 62: 25 training set model performance on the full testing set.

As can be seen from the last row of Table 8, when the training set increased to 100 images, the training time increased dramatically. The increased dataset pushes the CUDA RAM demand over 24GB (the maximum of RTX 4090), consequently the usage of shared GPU memory makes the process extremely slow. Coreset subsampling iteratively searches for the nearest neighbors from an testing image, and collects the farthest of which into the Coreset. The load grows significantly as the coreset expands, and is likely to occupy the whole of CUDA RAM when the coreset is large enough. After which waiting time would go beyond acceptance. Taking into account the up to standard performance and the reoccurring input, perhaps reducing the Coreset subsampling per-

centage(currently 0.5%) is a good idea to relieve the RAM pressure while maintaining good performances.

5.7 Limitations of Model Learning

While most overlaid heatmaps neatly highlight the flawed regions, a few instances exist in which the model fails to mark certain defects. For example in Figure 63 the HIP defect is not highlighted.



Figure 63: Unmarked HIP defect example.

This is because in the training set there happens to be a number of chips with flaws at the same position, the model recognized this existence in the training set and chose to leave that out. Example of one training set HIP defect in the same location is shown in Figure 64.



Figure 64: Training set chip with HIP defect at the same location.

It is worth noting that the unalarmed part in the image is not an anomaly, but is concerning when it is not signaled for human inspection. This can be fixed by removing those HIP defected chips in the training set, at the cost of raising more alarms on those non-anomaly regions. This decision needs to be properly consulted and evaluated, whether to improve automation for the production process, or to leave no room for the model to bail any defects.

6 Conclusions

Anomaly detection by unsupervised machine learning methods, proves to be effective in providing visual reference. The heatmap output, not only marks where it is likely

to be anomalous, but also gives different degrees of heat corresponding to the defective levels. Limitations from supervised methods such as: unlabeled undiscovered defects, poorly labeled anomalous regions, limited amount of anomaly training data are avoided or better dealt with by unsupervised methods.

Unlike supervised methods that demands pixel level ground truth, unsupervised methods are conducted on image-wise labeled input, making data collection much easier. Unsupervised learning methods that learn only the good samples, better differentiate the changes in the product and provide new perspective in picking out potential anomalies. Only a smaller subset of the normal images are involved in training which limits the methods' chance of over-fitting and strengthens its knowledge on the selected samples. However, as there is no specific labeling of what is the anomaly, the model is unable to ignore trivial visual differences or becomes over sensitive with tolerable artifacts. Cases that can be easily rule out with basic fluidics knowledge, becomes impossible for the model.

Beginning with the 3D point cloud, we conclude with 2D heatmaps where likely anomalous regions are marked. The unsupervised machine learning method proves to be effective for industrial anomaly detections. Learning only the passed samples and is therefore conservative, unsupervised methods work very well in a field that requires extra caution and care.

Embedded methods has shown their convenience of requiring no training, and their sensitivity to subtle defects. By making full use of the embedded features, PatchCore is able to differentiate bad or worst parts of the chip with different level of concerns, which further helped us in setting up a rule based mechanism. In conclusion our rules managed to sort out 21.2% chips from the whole testing set. With 1.5% error rate(kept bad chips) on the good chip sorting and 20.89% error(falsely rejected chips) rate on the bad chip sorting. It is far from perfect, but it is a small step towards final automation of microfluidics chip productions. Heatmaps can be good references for the operators.

Last but not the least, two new deformations which are not easily noticeable by human eyes are found with the help of PatchCore. Few failed chips which seems anomaly free are also discovered. These new discoveries hint at the usefulness of the method, which completes the argument of how good the state of the art methods are.

7 Future Work

Future work of this project can extend to different directions. Whether to incorporate into the industrial settings or to polish better the method. Depending on the demand, one can have different ideas on how to proceed. In the following section we elaborate a few different aspects.

Step-wise Incorporate into Productions To incorporate the sorting rules into production, one needs to specify an experienced operator to check especially the borderline chips(chips sorted out with threshold values). One must make sure that those chips are fine, and so will the chips that are largely tolerated by the threshold. For the

large anomaly rule, the smaller of which should be checked by the operator, verify that the threshold set for the rule is not too inclusive of the HIP defects. For sorting out the good chips, one should check the chips with its maximum anomaly value closest to the threshold, to ascertain that the threshold is not too high to include too many false negatives. Thresholds can then increase gradually, and hopefully converge under acceptable loss rates.

Post Sorting Analysis If sorting is done periodically like monthly, it would also be interesting to do distribution statistics of the good or bad chips through that month. Perhaps it can reveal how the production quality changes and give good suggestions on what to focus on in the next period. Figure 65 shows the number of chips selected by the maximum anomaly score rule by different dates along with total productions. Anomalies' positions can also be summarized, for example which section of the chip is more likely to have anomalies. Statistical analysis of the anomalies' spacial and time distribution may tell something about the productions. Checking the false alarms using a looser threshold may also be inspiring for productions, as there is so far no record for the number of HIP defects or Shim defects. The increase or decline of those defects are concerning, as those persisting false alarms cause distraction to operators, if not damage the chip. Which would in turn provide ground for real anomalies to be overlooked. The statistics on these visible distractions could help evaluate whether it is worthwhile to remove them.

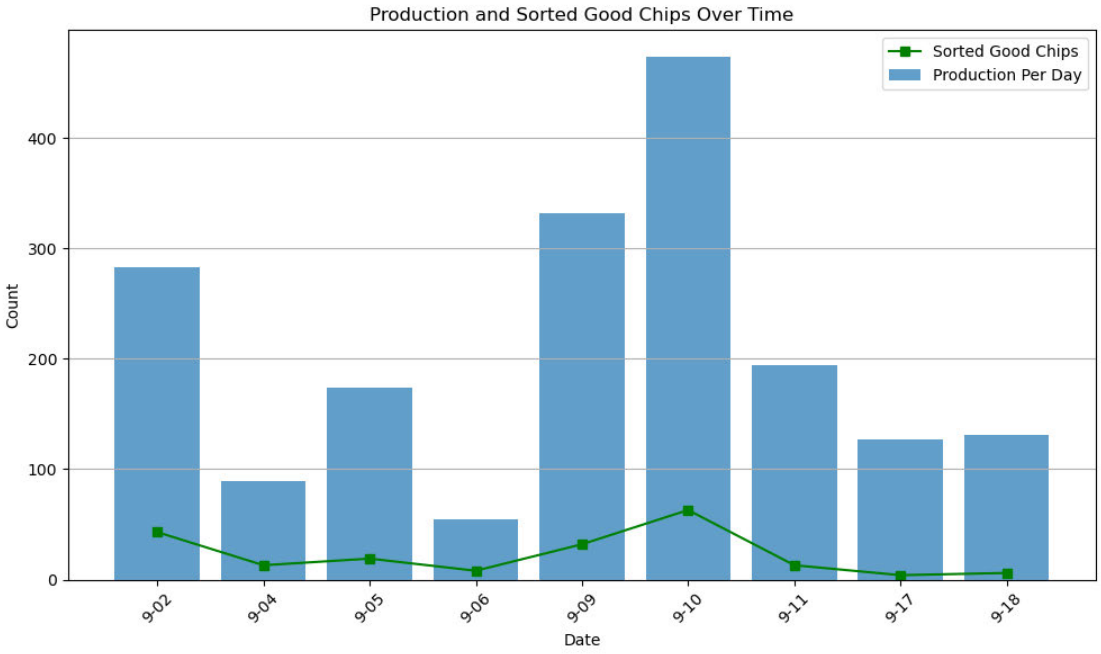


Figure 65: Good chips sorted by model vs daily productions.

Preprocessing Optimization Chip preprocessing needs to consider the need of including the peripheral lines, which are indications of possible leakage. This means that the cutting of the 3D point cloud needs to be redone. Chip coordination normalization was wrongly done in the prequel project, which causes the chip to shrink at different levels. This needs to be adjusted carefully. As the following Hough transform and seg-

mentations are closely connected to how the point cloud is projected into 2D. The 2D image's width and height and Hough radius need to be adjusted accordingly.

Embedded Method Related The PatchCore method is optimized by the FAPM[33] method, where the memory bank is updated with patch-wise and layer-wise memory banks, which is assumed to improve the inference speed as each patch would calculate the distance from vectors of that patch. It claims to be twice as fast as PatchCore and is worth trying out. The newly proposed embedded method HETMM[11] is also worth trying. Better localization method like DRAEM[15] would be fun to try out, but extra time need to be invested in classical training for this particular method.

Sorting Rules' Development The 3 rules we have used are still worthy of polishing. For example, the good chip rule which checks the maximum of the heatmap, could also be improved with a variance threshold, checking how much variance there is in the heatmap. This would be able to filter out a few chips with sub-significant error regions on the chip. Rules to sort out bad chips might also be augmented. If we are able to algorithmically differentiate HIP defect and Shim defect, then all other false alarms will be risky defects that have higher rate of hampering the chip, then it may be acceptable to discard all of the detected ones. Perhaps a model can be applied to negate the false alarms, for example: to say that this heated region is not to be evaluated, which should work well to remove the Shim defect false alarms. The challenge here is the varying size of the HIP defect, how to properly segment them for the model. New rules to sort should also be investigated, if there are better but more sophisticated indications than anomaly scores.

Adding A Production "Fail by Loss" Button As mentioned in Section 5.5.2 we have controversial chips which are failed through FocalSpec system. These chips are not anomalous chips but are chips that were lost during production. Unfortunately these chips are likely to be picked out by the good chip rule, and taken as false negatives(bad chips that got tolerated by the rule). As we are having a tight restriction on falsely approve bad chips, these 1-2 chips significantly lower the good rule sorting metrics, causing the rule to down perform by 0.5%.

References

[1] P. Pattanayak, S.K. Singh, M. Gulati, et al. Microfluidic chips: Recent advances, critical strategies in design, applications and future perspectives. *Microfluidics and Nanofluidics*, 25, 10 2021.

[2] LMI Technologies. FocalSpec high speed line confocal sensors. Available: url https://lmi3d.com/focalspec-high-speed-line-confocal-sensors/.

[3] Christoph Baur, Benedikt Wiestler, Shadi Albarqouni, and Nassir Navab. Deep autoencoding models for unsupervised anomaly segmentation in brain mr images. In K. Zhang, A. Martel, D. Shen, S. Zuluaga, D. Racoceanu, and L. Joskowicz, editors, *Lecture Notes in Computer Science*, volume 11383 of *LNCS*, pages 161–169. Springer, 2018.

[4] Guodong Wang, Shumin Han, Errui Ding, and Di Huang. Student-teacher feature pyramid matching for anomaly detection. In *The 32nd British Machine Vision Conference (BMVC)*, page 306. BMVA Press, 2021.

[5] Hanqiu Deng and Xingyu Li. Anomaly detection via reverse distillation from one-class embedding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9737–9746, June 2022.

[6] Samet Akcay, Amir Atapour-Abarghouei, and Toby P. Breckon. GANomaly: Semi-supervised anomaly detection via adversarial training. In Chin-Hui Lee et al., editors, *Computer Vision – ACCV 2018*, volume 11363 of *Lecture Notes in Computer Science*, pages 622–637. Springer, Cham, 2019.

[7] Niv Cohen and Yedid Hoshen. Sub-image anomaly detection with deep pyramid correspondences. *arXiv preprint arXiv:2005.02357*, May 2020. Available at https://arxiv.org/abs/2005.02357.

[8] Thomas Defard, Aleksandr Setkov, Angelique Loesch, and Romaric Audigier. PaDim: A patch distribution modeling framework for anomaly detection and localization. In *Pattern Recognition. ICPR International Workshops and Challenges: Virtual Event, January 10–15, 2021, Proceedings, Part IV*, page 475–489, Berlin, Heidelberg, 2021. Springer-Verlag.

[9] Karsten Roth, Latha Pemula, Joaquin Zepeda, Bernhard Schölkopf, Thomas Brox, and Peter Gehler. Towards total recall in industrial anomaly detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14298–14308, New Orleans, USA, June 2022. DOI: 10.1109/CVPR52688.2022.01392.

[10] Sungwook Lee, Seunghyun Lee, and Byung Cheol Song. CFA: Coupled-hypersphere-based feature adaptation for target-oriented anomaly localization. *IEEE Access*, 10:78446–78454, July 2022. DOI: 10.1109/ACCESS.2022.3193699.

[11] Zixuan Chen, Xiaohua Xie, Lingxiao Yang, and Jianhuang Lai. Hard-normal example-aware template mutual matching for industrial anomaly detection. *International Journal of Computer Vision*, 133(5):2927–2949, 2025.

[12] Denis Gudovskiy, Shun Ishizaka, and Kazuki Kozuka. CFLOW-AD: Real-time unsupervised anomaly detection with localization via conditional normalizing flows. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2022.

[13] Jiawei Yu, Ye Zheng, Xiang Wang, Wei Li, Yushuang Wu, Rui Zhao, and Liwei Wu. FastFlow: Unsupervised anomaly detection and localization via 2d normalizing flows. *arXiv*, 2021.

[14] Paul Bergmann, Michael Fauser, David Sattlegger, and Carsten Steger. Uninformed students: Student–teacher anomaly detection with discriminative latent embeddings. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4182–4191, June 2020.

[15] Vitjan Zavrtanik, Matej Kristan, and Danijel Skočaj. DRAEM: A discriminatively trained reconstruction embedding for surface anomaly detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 8330–8339, Montreal, Canada, October 2021. Also available on arXiv: arXiv:2108.07610.

[16] Chun-Liang Li, Kihyuk Sohn, Jinsung Yoon, and Tomas Pfister. CutPaste: Self-supervised learning for anomaly detection and localization. In *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9659–9669, 06 2021.

[17] Nicolae-Catalin Ristea, Neelu Madan, Radu Tudor Ionescu, Kamal Nasrollahi, Fahad Shahbaz Khan, Thomas B. Moeslund, and Mubarak Shah. Self-supervised predictive convolutional attentive block for anomaly detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 13576–13586, 2022.

[18] Neelu Madan, Nicolae-Catalin Ristea, Radu Tudor Ionescu, Kamal Nasrollahi, Fahad Shahbaz Khan, Thomas B. Moeslund, and Mubarak Shah. Self-supervised masked convolutional transformer block for anomaly detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 46(1):525–542, 2024.

[19] Zhikang Liu, Yiming Zhou, Yuansheng Xu, and Zilei Wang. SimpleNet: A simple network for image anomaly detection and localization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 20402–20411, 2023.

[20] Qiyu Chen, Huiyuan Luo, Han Gao, Chengkan Lv, and Zhengtao Zhang. Progressive boundary guided anomaly synthesis for industrial anomaly detection. *IEEE Transactions on Circuits and Systems for Video Technology*, 35(2):1193–1208, 2025.

[21] Qiyu Chen, Huiyuan Luo, Chengkan Lv, and Zhengtao Zhang. A unified anomaly synthesis strategy with gradient ascent for industrial anomaly detection and localization. In *Proceedings of the European Conference on Computer Vision (ECCV)*, volume 15125 of *Lecture Notes in Computer Science*, pages 37–54. Springer, 2024.

[22] Kilian Batzner, Lars Heckler, and Rebecca König. EfficientAD: Accurate visual anomaly detection at millisecond-level latencies. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 128–138, 2024.

[23] Xuan Zhang, Shiyu Li, Xi Li, Ping Huang, Jiulong Shan, and Ting Chen. DeSTSeg: Segmentation guided denoising student-teacher for anomaly detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3914–3923, 2023.

[24] Haoyang He, Yuhu Bai, Jiangning Zhang, Qingdong He, Hongxu Chen, Zhenye Gan, Chengjie Wang, Xiangtai Li, Guanzhong Tian, and Lei Xie. MambaAD: exploring state space models for multi-class unsupervised anomaly detection. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, NIPS '24, pages 71162 – 71187, Red Hook, NY, USA, 2025. Curran Associates Inc.

[25] Haoyang He, Jiangning Zhang, Hongxu Chen, Xuhai Chen, Zhishan Li, Xu Chen, Yabiao Wang, Chengjie Wang, and Lei Xie. DiAD: A diffusion-based framework for multi-class anomaly detection. In *Proceedings of the Thirty-Eighth AAAI Conference on Artificial Intelligence (AAAI)*, pages 8472–8480, 2024.

[26] Wei Luo, Yunkang Cao, Haiming Yao, Xiaotian Zhang, Jianan Lou, Yuqi Cheng, Weiming Shen, and Wenyong Yu. Exploring intrinsic normal prototypes within a single image for universal anomaly detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9974–9983, June 2025.

[27] Zhiyuan You, Lei Cui, Yujun Shen, Kai Yang, Xin Lu, Yu Zheng, and Xinyi Le. A unified model for multi-class anomaly detection. In *Proceedings of the 36th Conference on Neural Information Processing Systems (NeurIPS)*, pages 4571 – 4584, 2022.

[28] John Canny. A computational approach to edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-8(6):679–698, 1986.

[29] Lars Heckler, Rebecca König, and Paul Bergmann. Exploring the importance of pretrained feature extractors for unsupervised anomaly detection and localization. In *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 2917–2926, 2023.

[30] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The Faiss library. 2024.

[31] Samet Akcay, Dick Ameln, Ashwin Vaidya, Barath Lakshmanan, Nilesh Ahuja, and Utku Genc. Anomalib: A deep learning library for anomaly detection. In *2022 IEEE International Conference on Image Processing (ICIP)*, pages 1706–1710. IEEE, 2022.

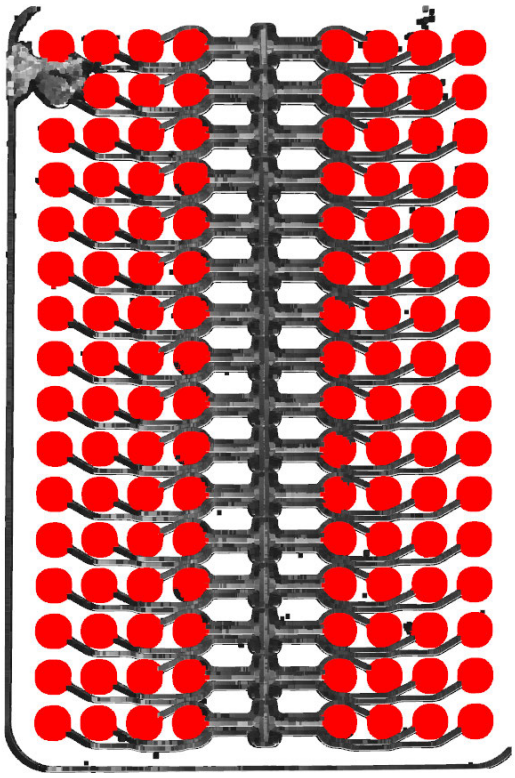
[32] Alessandro Peratoner. AutoML for advanced monitoring in digital manufacturing and industry 4.0. Master's thesis, University of Padua, Department of Information Engineering (DEI), Padua, Italy, February 2023. Supervisor: Gian Antonio Susto.

[33] Donghyeong Kim, Chaewon Park, Suhwan Cho, and Sangyoun Lee. FAPM: Fast adaptive patch memory for real-time industrial anomaly detection. In *ICASSP 2023 - 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 1–5, June 2023.

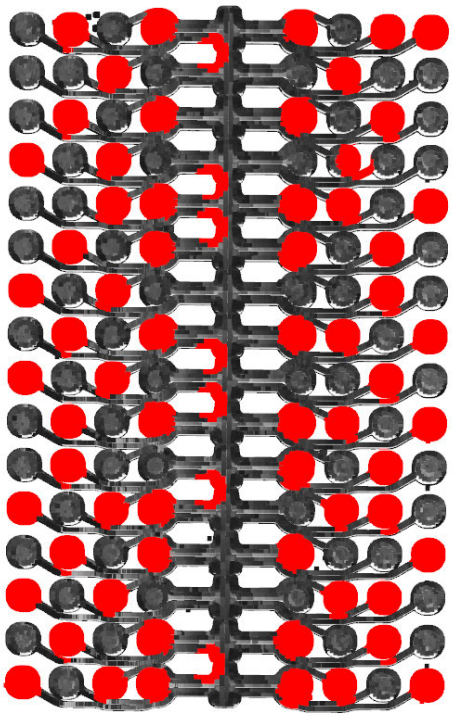
A Appendix

A.1 Double Hough Missing Chips

There were only 2 chips where our double Hough methods failed. One(3302-15-E3) due to having a huge defect around the circles, causing the circle edge to be defected, as shown in Figure 66a. The other(3299-5-A4) chip's result show a random segmentation of circles as shown in Figure 66b.



(a) Large region defected chip(3302-15-E3) Hough result.



(b) "Small" chip(3299-5-A4) Hough result

Figure 66: Failed double Hough segmentations.

After careful examining the 3D point cloud, it is evident that this image's circles are much smaller and has led to this failure in segmentation. The cause of the "shrinking" is related with the preprocessing and normalization of coordinates, which caused that particular chip to be smaller than normal. We can also see extra chip structures in 2D projections in Figure 67 due to the chip being shrunken in size. Consequently the Hough segmentation radius was too large ending up losing large number of circles. The 2 chips are displayed in Figure 66. The coordinate normalization was removed at the end of the project in mid August. Following sections' measurement unit has also changed to "point cloud unit" instead of Hough radius unit.

(a) Large region defected chip(3302-15-E3).

(b) ”Small” chip(3299-5-A4).

Figure 67: Double Hough unsuccessful chip originals.

A.2 Row and Column Width Distribution

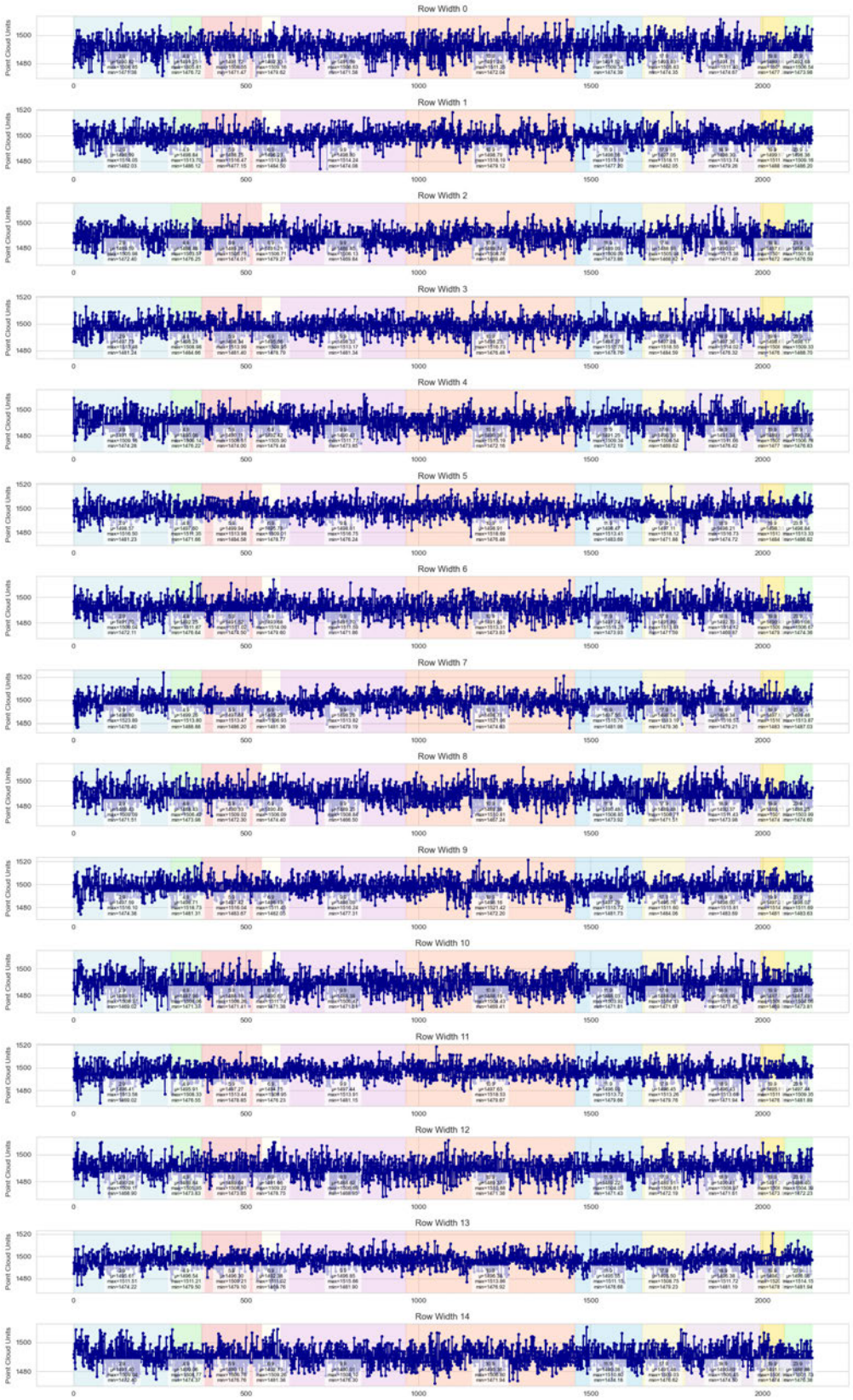


Figure 68: Row width along time.



Figure 69: Column width along time.

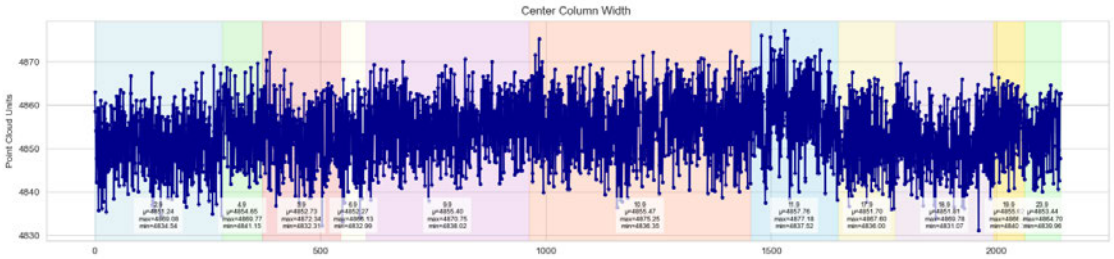


Figure 70: Center column width across time.

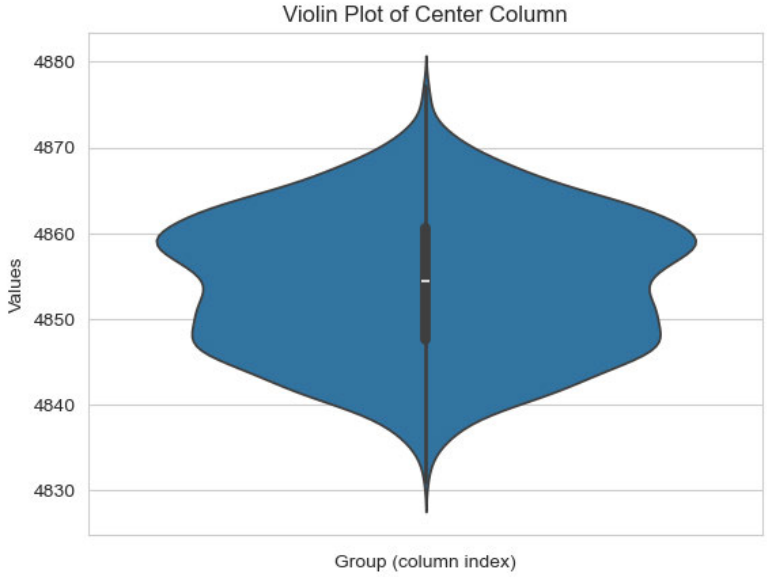


Figure 71: Center column width violin plot.

A.3 Exploration on Heatmap Visualizations

After normalizing the heatmaps by global min-max values, various color maps can be applied. There are 86 different colormaps in the Matplotlib library, add its reversed version would double the options to 172. Commonly used one is the ”jet” color map which is very sharp and known to be biased. We’ve tried out a few color maps including: ”hot”, ”inferno”, ”twilight”, ”gnuplot2”, ”cubehelix” and ”viridis”(close to the parula colormap in Matlab). Images of which are shown in Figure 72 to 74.

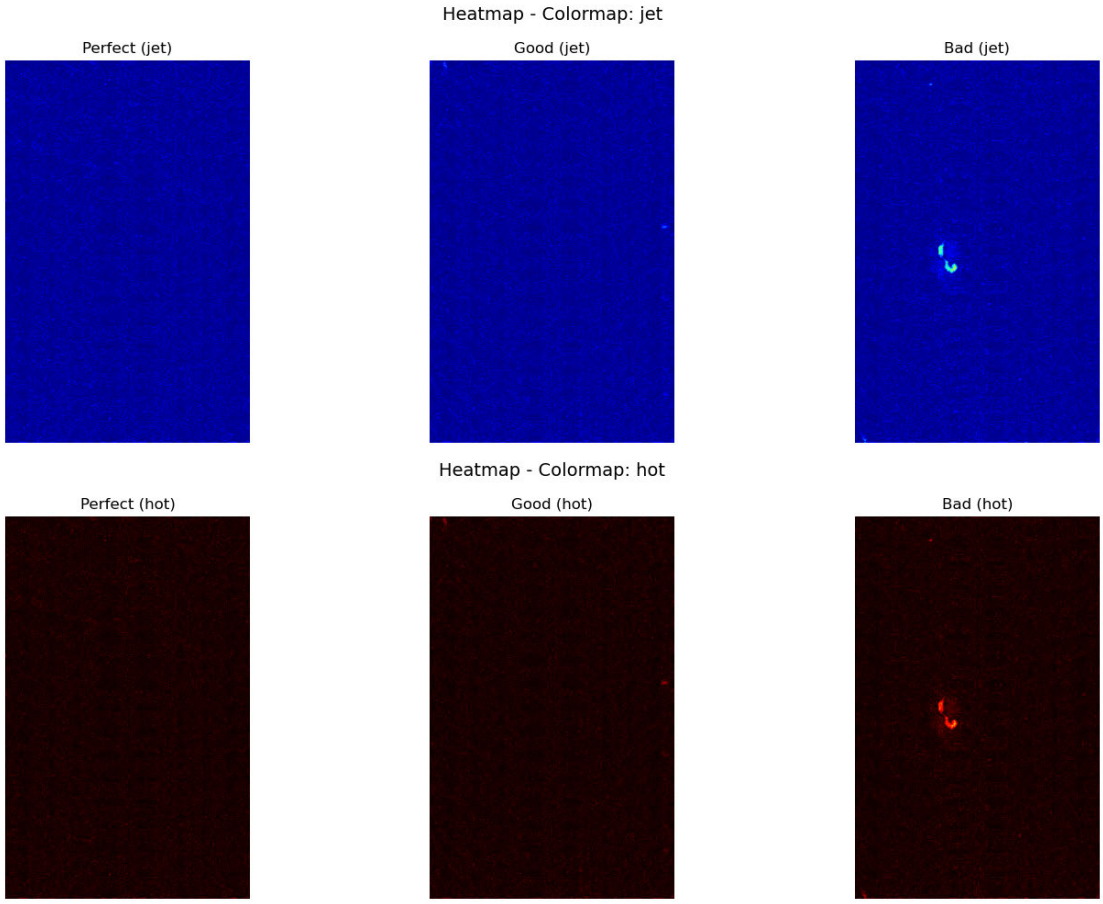


Figure 72: Colormap visualizations (jet and hot).

”Viridis” though less saturated than the ”jet” colormap, is a bit vague in showing the artifacts. A different colored background is also less preferred, as the chip itself is black on the basis part, a different color can be distracting. So we’ve tried out more colormaps with black background, from ”cubehelix” to ”inferno”. In comparison, the center good image is best for checking visualization effect. Because the perfect chips have nothing to look at and the worst chip is well stressed in all colormaps.

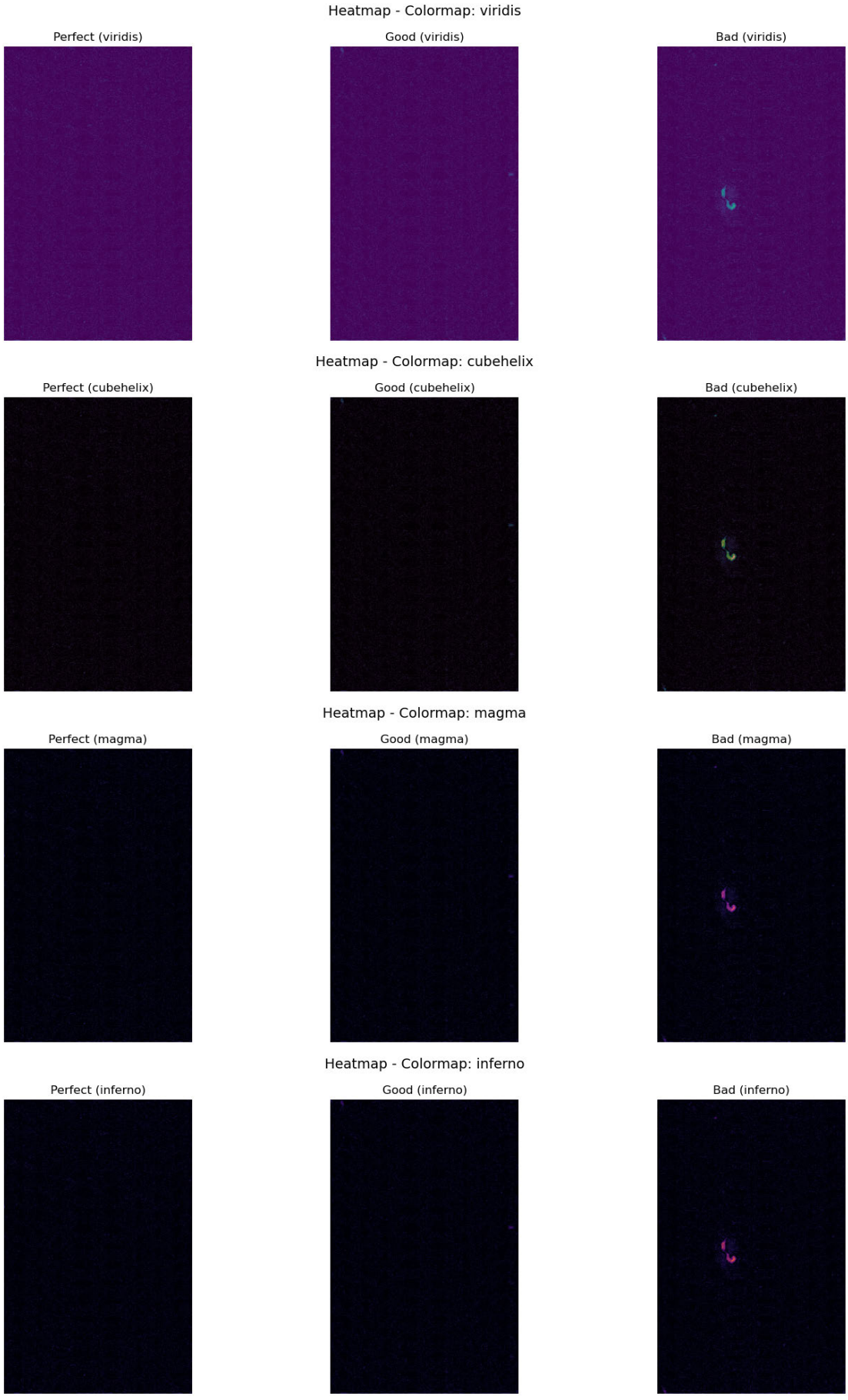
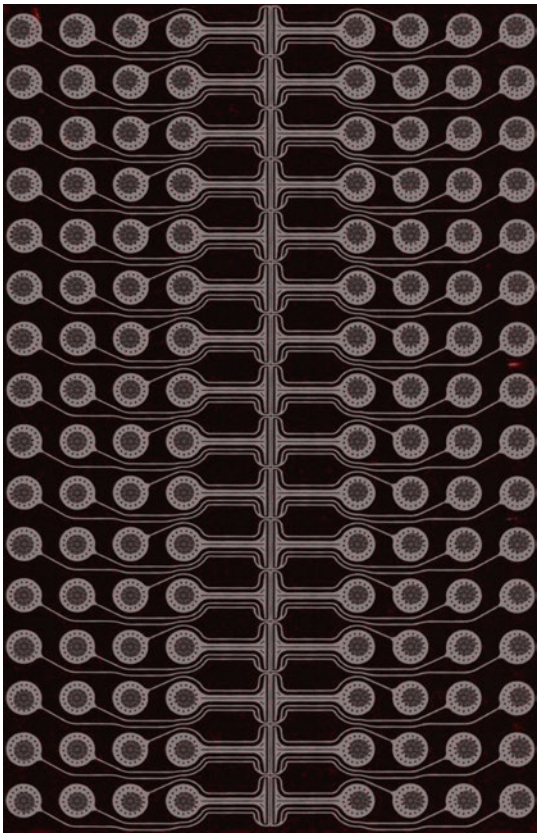


Figure 73: Colormap visualization (4 of 7).

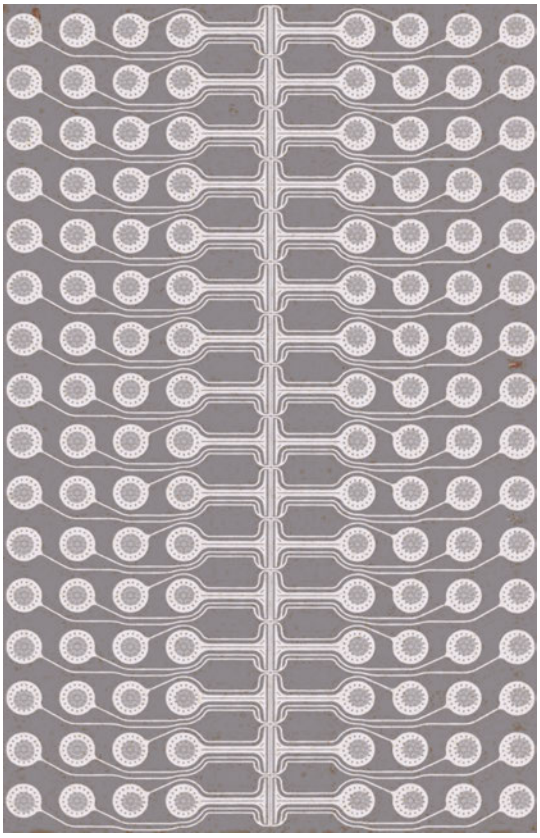


Figure 74: Colormap visualization (7 of 7).

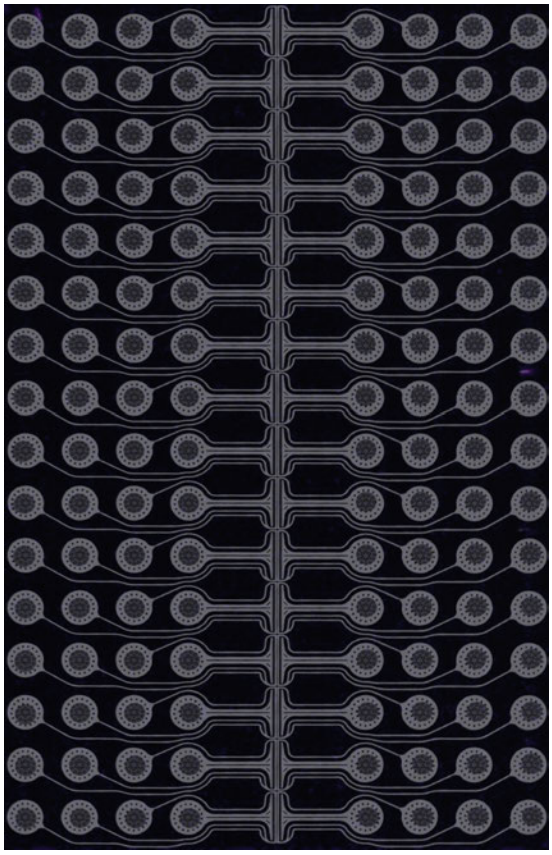
It is rather difficult to decide what is best for visualization, since one person can only say for oneself. By assumption it is better to introduce less new colors to the chip, to avoid bias and help better focus on where is marked. Below Figure 76 displays 6 overlays with either black or white as background. From our point of view the best 2 are the ”hot” and ”twilight_r”. ”Hot” is darker and makes good differentiation on the artifacts. In comparison ”twilight_r” is brighter, better stress what is marked on the white channels but has a large number of sensitive points across the heatmap.



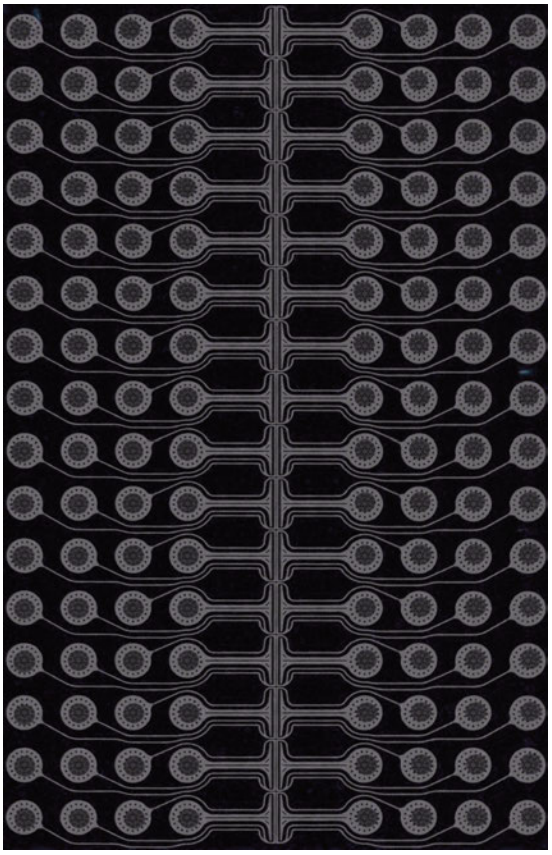
(a) Hot colormap overlay(anomalies in red in upper left corner and center right).



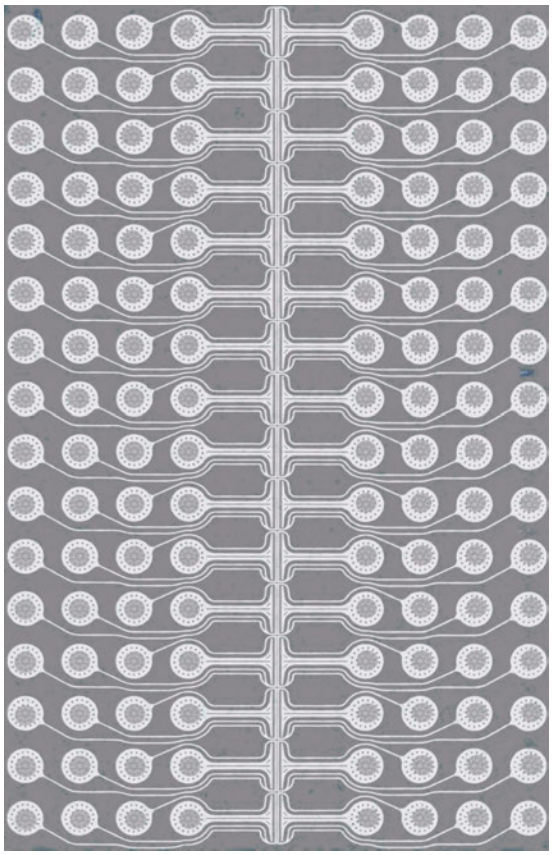
(b) Twilight_r colormap overlay(anomalies in light red in upper left corner and center right).



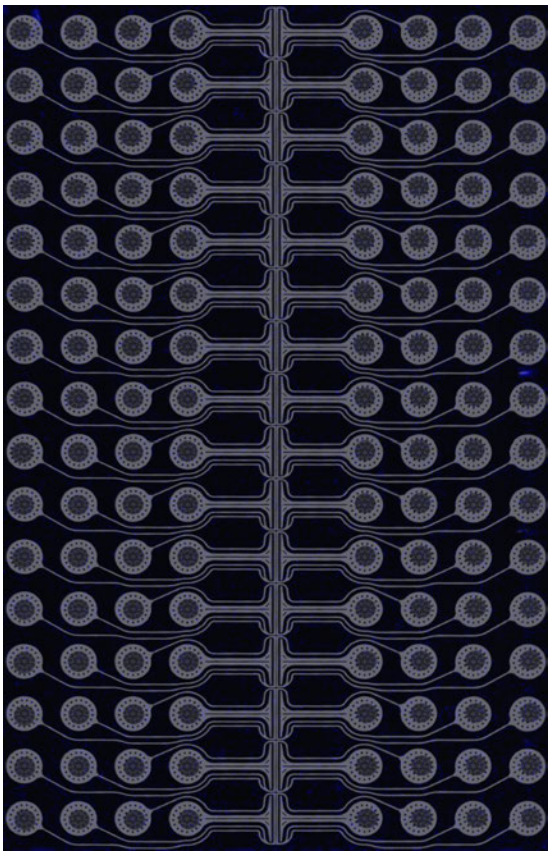
(c) Inferno colormap overlay(anomalies in purple).



(d) Cubehelix colormap overlay(anomalies in emerald).



(e) Twilight colormap overlay(anomalies in light blue).



(f) Gnuplot2 colormap overlay(anomalies in dark blue).

Figure 76: Colormap overlay visualizations.

Another idea is to stress the contrast between the background and the anomaly. In this sense the background is not necessary to remain the same as long as the ”heated” parts are well stressed. Colormaps picked under this philosophy are ”brg” and ”gist_rainbow”, illustrated in Figure 77. But it may be exhausting to look at for long time, like going through hundreds of images per day.

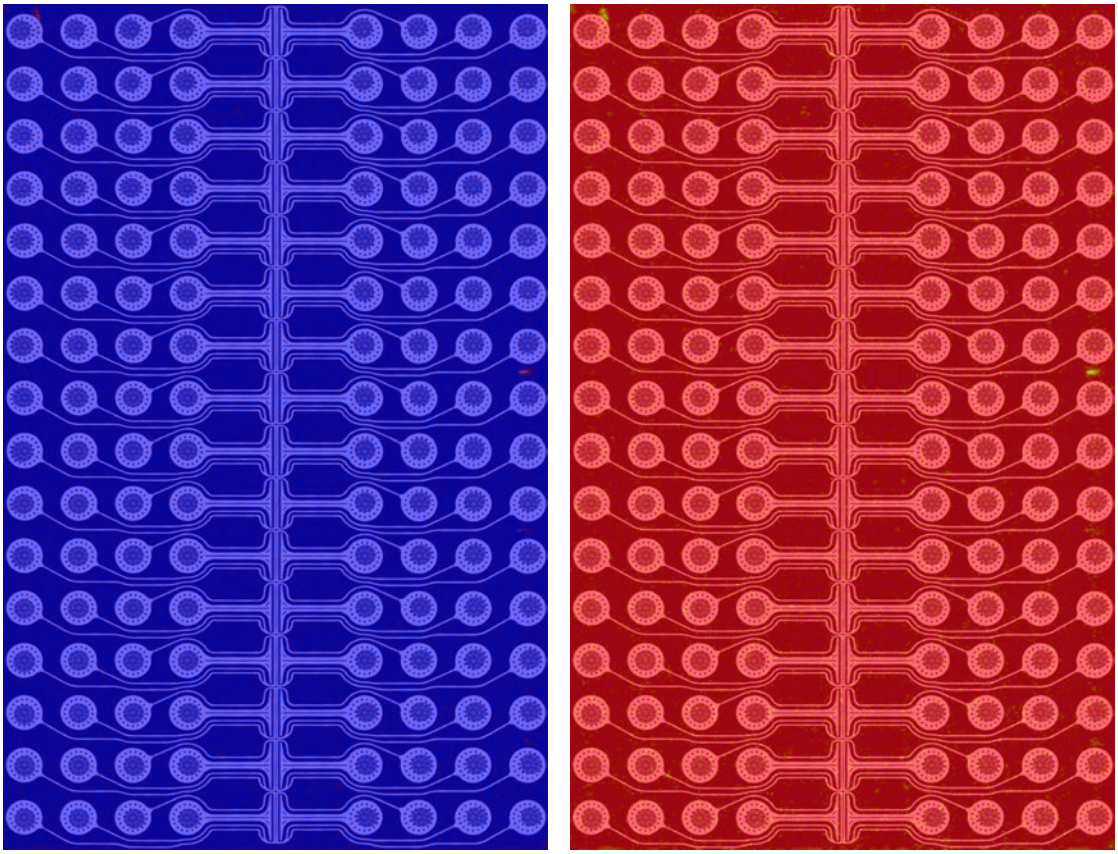


Figure 77: Overlay of ”gist_rainbow” and ”brg”, spots in the center right and upper left corner are easily visible.

An interesting type of colormaps are the qualitative ones, which have a discrete spectrum in intensities. This provides better localization of the anomalies as there is less transition between good and bad pixels. The colormaps' transitions are shown in Figure 78.

Several colormaps of this category provide good visualizations, such as the two illustrated in Figure 79 and Figure 80 . Depending on the periodicity length, some are more sensitive with isolated pixels like ”Paired”, ”tab10”, ”tab10_r”. Less sensitive ones are ”Dark_r”, ”Accent_r”, ”Set1_r” and ”Set2_r”.

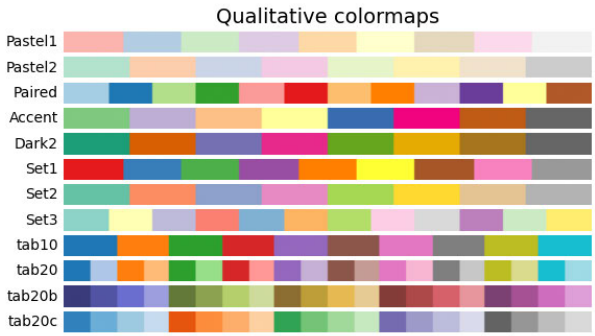
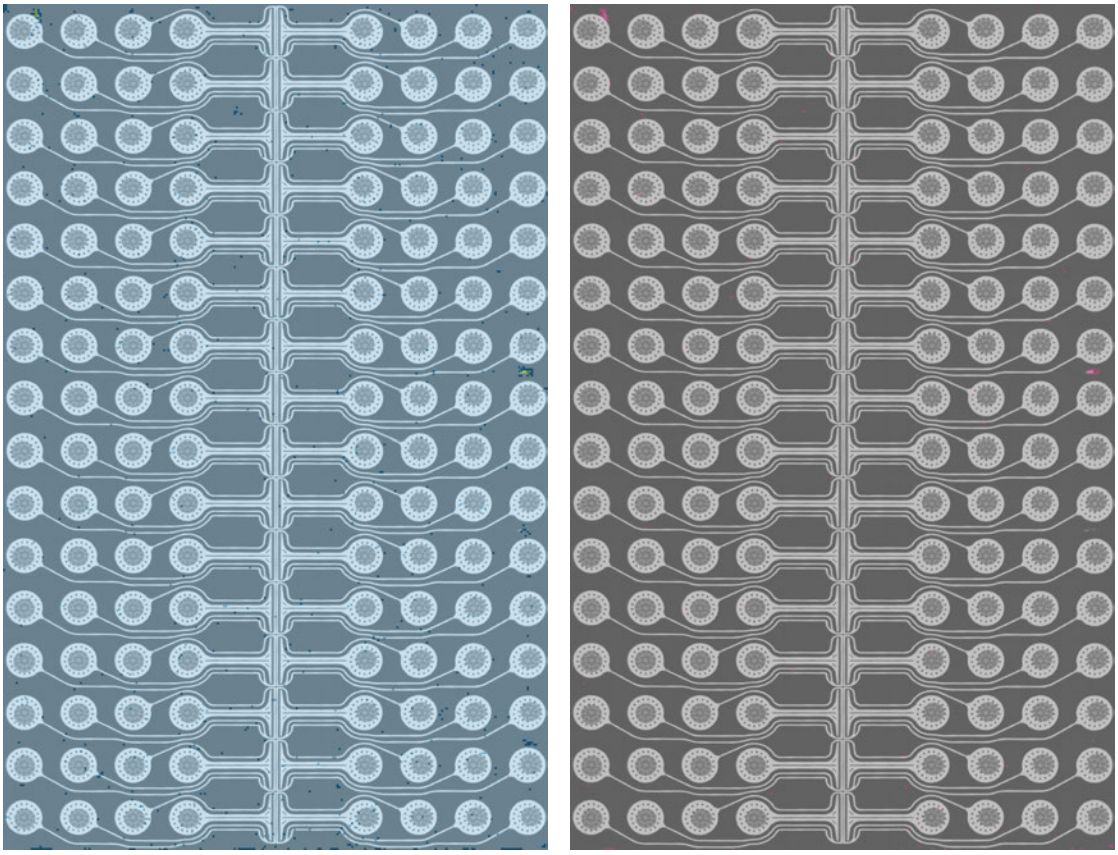


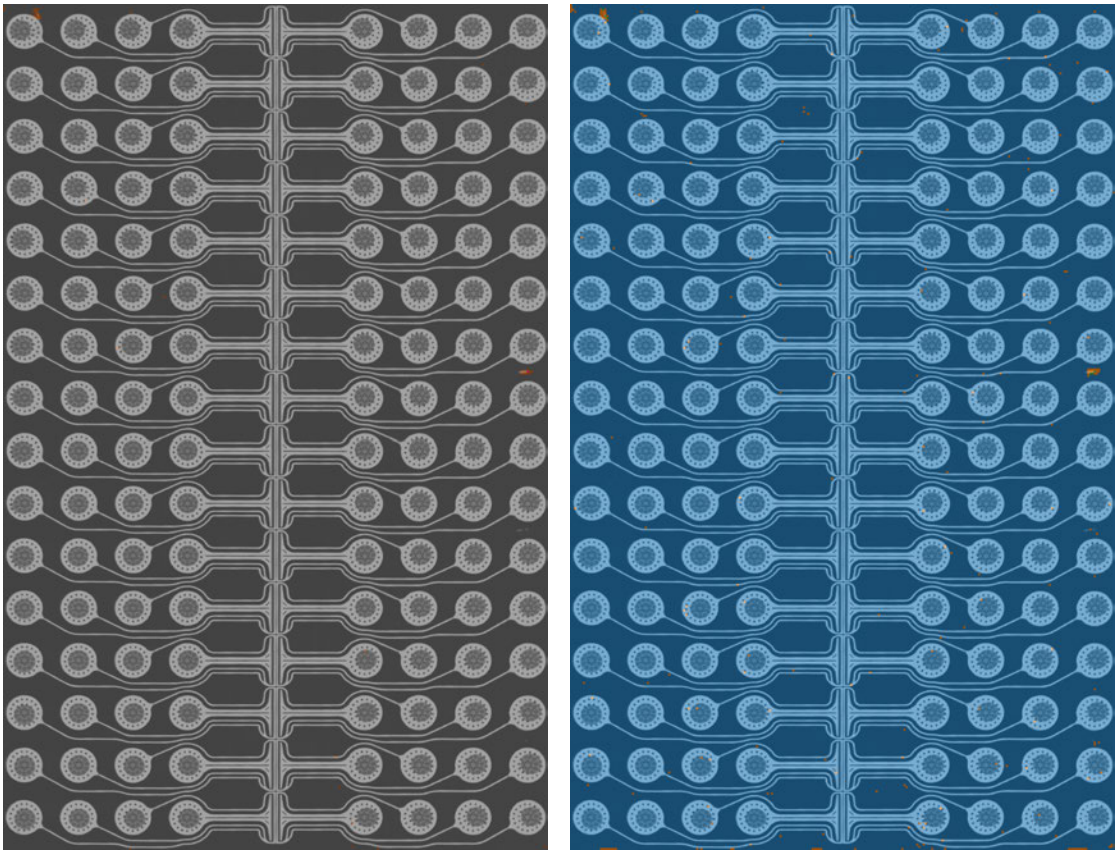
Figure 78: Qualitative colormaps, showing different periodicity.



(a) Overlay with ”Paired” colormap.

(b) Overlay with ”Set1_r” colormap.

Figure 79: Examples of qualitative colormaps, the ”Paired” on the left is more pixeled and too sensitive compared with ”Set1_r” on the right where pinkish colors are used for heat spots.



(a) Overlay with ”Accent_r” colormap. (b) Overlay with ”tab10” Colormap.

Figure 80: Comparison of color map ”Accent_r” and ”tab10”, ”tab10” showing more pixels due to sensitivity, heat spots in the center right and upper left are well marked in both images.

Consider that colormaps are perceived differently, in future implementations it is suggested to have multiple choice of colormaps. So that different people can select the one that is most preferred and comfortable. Unfortunately, in current working environment there is no color blind persons to reference from, so the effectiveness of these colormaps are yet to be fully tested.



Figure 81: 3 Pixels with largest intensity in the heatmaps, the perfect image has 3 highest points separated in the image while the other 2 are focused on the flawed part.

Another way to visualize the most anomalous parts is to mark the points with largest intensities in the heatmap. Finding the largest 3 points and mark those with red squares, the original regions can be found correspondingly as illustrated in Figure 82.

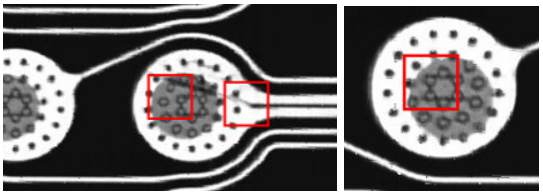


Figure 82: Localization of the 3 squares from the perfect image.

The marked 3 squares of the perfect image are revealing, as they are marked on areas different than normal images. Left image of Figure 82 shows harmless scratches on the chips while the right image shows a slightly drifted inner circle rim. From the center image of Figure 81 we can see the limitation of this marking method. Only the center right flaw is marked, the upper left defect mentioned in previous text is not marked, because the value there is smaller than the center right flaw.